



SPOT WARSAW GUIDE

Inteligentny system rekomendacji miejsc w Warszawie

DOKUMENTACJA TECHNICZNA

Projekt	Technologie
Warsaw Rec — RAG System + Mobile App	FastAPI · React Native · PostgreSQL · TimescaleDB · OpenAI · AWS
Rok	2026
Autorzy	Maria Pacocha, Magdalena Pacocha

Spis treści

1. Opis projektu

- 1.1 Cel i idea
- 1.2 Zakres funkcjonalny
- 1.3 Użyte technologie

2. Architektura systemu

- 2.1 Przegląd architektury
- 2.2 Komunikacja między warstwami
- 2.3 RAG Pipeline — przepływ danych
- 2.4 Przepływ autentykacji
- 2.5 Zewnętrzne usługi i integracje
- 2.6 Backend — struktura i komponenty
- 2.7 Frontend — struktura i komponenty
- 2.8 Patch biblioteki react-native-maps

3. Dane — źródła, przetwarzanie i struktura wektorowa

- 3.1 Źródło danych — Google Places API
- 3.2 Pipeline przetwarzania danych
- 3.3 Struktura wektorowa i wyszukiwanie semantyczne

4. Schemat bazy danych

- 4.1 Tabele
- 4.2 Tabela embeddings — kluczowe pola
- 4.3 Tabela saved_places — schemat kolumn
- 4.4 Tabela chat_sessions
- 4.5 Tabela chat_messages

5. API — Endpointy

- 5.1 Autoryzacja
- 5.2 Chat
- 5.3 Miejsca
- 5.4 Sesje

6. Obsługa błędów i fallbacki

- 6.1 Backend — kody odpowiedzi i zachowanie
- 6.2 Frontend — obsługa błędów po stronie klienta
- 6.3 Limity zewnętrznych usług

7. Diagramy przepływu

- 7.1 Przepływ wiadomości czatu
- 7.2 Przepływ autentykacji

8. Instrukcja uruchomienia — lokalnie

- 8.1 Wymagania wstępne
- 8.2 Konfiguracja środowiska
- 8.3 Uruchomienie bazy danych
- 8.4 Pobranie i załadowanie danych

- 8.5 Provisioning AWS (Terraform)
- 8.6 Uruchomienie backendu
- 8.7 Uruchomienie frontendu

9. Konfiguracja i parametry

- 9.1 Parametry modeli AI
- 9.2 Zmienne środowiskowe

10. Znane ograniczenia i uwagi

- 10.1 Ograniczenia techniczne
- 10.2 Koszty API
- 10.3 Bezpieczeństwo — odesłanie

11. Testy manualne — potwierdzenie działania funkcji

12. Architektura infrastruktury i strategia wdrożenia chmurowego

- 12.1 Struktura katalogów Terraform
- 12.2 Konfiguracja konta AWS
- 12.3 Inicjalizacja i uruchomienie Terraform
- 12.4 Sieć — VPC i subnety
- 12.5 Domena i DNS — Route 53
- 12.6 Certyfikat SSL — ACM
- 12.7 Application Load Balancer
- 12.8 ECS Fargate — środowisko uruchomieniowe
- 12.9 Moduł ga — OIDC i rola IAM dla GitHub Actions
- 12.10 Dockerfile — konfiguracja kontenera
- 12.11 Zmienne środowiskowe — strategia dla środowisk
- 12.12 Migracja bazy danych z lokalnego TimescaleDB na AWS RDS
- 12.13 Backup bazy danych
- 12.14 Monitoring i alarmy CloudWatch
- 12.15 IAM — zarządzanie uprawnieniami
- 12.16 Szacunkowe koszty infrastruktury

13. Środowisko developerskie a produkcyjne

- 13.1 Środowisko developerskie (lokalne)
- 13.2 Testowanie aplikacji mobilnej
- 13.3 Build produkcyjny
- 13.4 Deploy backendu na AWS
- 13.5 Weryfikacja wdrożenia

14. CI/CD — automatyczne wdrożenia przez GitHub Actions

- 14.1 Jak działa pipeline
- 14.2 OIDC — uwierzytelnienie do AWS
- 14.3 Cache Docker
- 14.4 Tagowanie i rollback

15. Troubleshooting

16. Bezpieczeństwo — OWASP Web, API i Mobile Top 10

- 16.1 Metodologia testowania
- 16.2 OWASP Web Top 10:2025 — kategorie i pokrycie
- 16.3 Mapowanie plików testowych na kategorie OWASP
- 16.4 Status pokrycia — OWASP Web Top 10:2025

- 16.5 OWASP API Top 10:2023 — pokrycie
- 16.6 Audyt zależności (A03 — Supply Chain)
- 16.7 OWASP Mobile Top 10:2024 — pokrycie
- 16.8 OWASP MASVS L1 — weryfikacja
- 16.9 Testy produkcyjne i poprawki wdrożeniowe
- 16.10 Wnioski z testów bezpieczeństwa
- 16.11 Podsumowanie

1. Opis projektu

1.1 Cel i idea

Warsaw Rec to conversational system rekomendacji miejsc gastronomicznych i kulturalnych w Warszawie. System łączy duże modele językowe (LLM) z wyszukiwaniem wektorowym (RAG — Retrieval Augmented Generation), aby dostarczać spersonalizowane, kontekstowe rekomendacje w oparciu o naturalny dialog z użytkownikiem.

Główna idea opiera się na obserwacji, że tradycyjne wyszukiwarki (np. Google Maps) wymagają od użytkownika precyzyjnego formułowania zapytań. Warsaw Spot Guide pozwala na swobodną rozmowę — użytkownik może napisać "szukam kawiarni z dobrą kawą na Mokotowie do pracy z laptopem z klimatycznym wnętrzem" i otrzymać trafne rekomendacje z wyjaśnieniem wyboru.

Spot Guide jest aplikacją mobilną na systemy iOS i Android, zbudowaną w React Native z Expo SDK 54. Użytkownik instaluje aplikację na telefonie i komunikuje się z systemem przez pięć głównych widoków: ekran czatu z asystentem AI, interaktywną mapę Warszawy z markerami miejsc, listę zapisanych ulubionych, ekran szczegółów miejsca oraz profil użytkownika. Autentykacja opiera się na AWS Cognito z pełnym cyklem rejestracji, weryfikacji email i odświeżania tokenów JWT.

1.2 Zakres funkcjonalny

Backend — RAG Pipeline

- Klasyfikacja wiadomości użytkownika (RAG / followup / hybrid)
- Ekstrakcja metadanych z zapytania (dzielnica, cena, kategoria, godziny)
- HyDE (Hypothetical Document Embedding) dla lepszej jakości wyszukiwania
- Wyszukiwanie wektorowe z predykatami w TimescaleDB / pgvector
- Synteza odpowiedzi przez LLM (gpt-5.4-mini) z kontekstem historii
- Przechowywanie historii sesji i ulubionych miejsc w PostgreSQL
- REST API (FastAPI) z autentykacją JWT (AWS Cognito)

Frontend — Aplikacja mobilna (iOS/Android)

- Ekran czatu z kartami poleconych miejsc w poziomym sliderze
- Historia sesji z grupowaniem po datach
- Mapa z markerami kategorii i clusteringiem
- Filtrowanie miejsc (kategoria, podkategoria, cena, dzielnica)
- Zapisane miejsca (ulubione) z synchronizacją do bazy
- Ekran szczegółów miejsca z godzinami, tagami i linkiem do Google Maps lub menu
- Profil użytkownika (AWS Cognito)

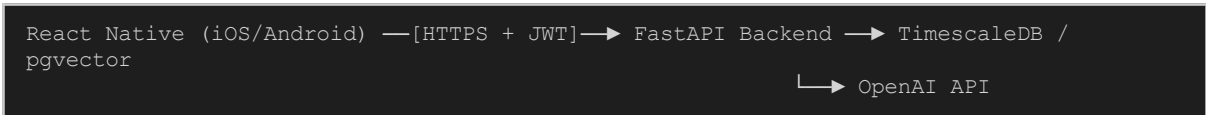
1.3 Użyte technologie

Warstwa	Technologia	Zastosowanie
AI / ML	OpenAI gpt-5.4-mini	Klasyfikacja, ekstrakcja, synteza odpowiedzi
AI / ML	text-embedding-3-small	Generowanie embeddingów do wyszukiwania
Baza danych	TimescaleDB / pgvector (PostgreSQL)	Wektory, embeddingi, historia czatu, miejsca
Backend	FastAPI (Python)	REST API, routing, walidacja, autoryzacja
Backend	Pydantic / Instructor	Walidacja danych, structured outputs z LLM
Frontend	React Native (Expo 54)	Aplikacja mobilna iOS i Android
Frontend	react-native-maps 1.20.1	Wyświetlanie mapy z markerami (patch wymagany)
Frontend	react-native-map-clustering	Clustering markerów na mapie
Frontend	Zustand	Zarządzanie stanem aplikacji
Auth	AWS Cognito	Rejestracja, logowanie, tokeny JWT
Infrastruktura	Docker / Docker Compose	Lokalny kontener z TimescaleDB
Infrastruktura	Terraform	Provisioning całej infrastruktury AWS (IaC) — VPC, ECS, RDS, ALB, Cognito
Infrastruktura	GitHub Actions	CI/CD — automatyczny deployment backendu na ECS

2. Architektura systemu

2.1 Przegląd architektury

System składa się z trzech głównych warstw: aplikacji mobilnej (frontend), serwera API (backend) oraz bazy danych. Komunikacja między frontendem a backendem odbywa się przez REST API z autoryzacją JWT. Backend komunikuje się z zewnętrznym API OpenAI oraz wewnętrzną bazą danych.



2.2 Komunikacja między warstwami

Aplikacja mobilna komunikuje się z backendem wyłącznie przez REST API zabezpieczone tokenami JWT. Każdy request HTTP zawiera nagłówek Authorization: Bearer <access_token>, który FastAPI weryfikuje przy użyciu publicznych kluczy JWKS pobieranych z endpointu AWS Cognito. Klucze są cache'owane po stronie backendu przez dekorator @lru_cache, dzięki czemu każdy request nie generuje dodatkowego wywołania sieciowego.

Backend komunikuje się z dwiema zewnętrznymi usługami: OpenAI API (klasyfikacja, ekstrakcja metadanych, HyDE, synteza odpowiedzi) oraz bazą danych PostgreSQL/pgvector (wyszukiwanie wektorowe, historia czatu, ulubione miejsca). Połączenie z bazą jest utrzymywane przez pulę połączeń psycopg2 — każdy request do bazy nie otwiera nowego połączenia.

Na produkcji cały ruch przechodzi przez Application Load Balancer, który terminuje TLS i przekazuje niezaszyfrowany ruch HTTP do kontenera ECS wewnątrz prywatnej sieci VPC. Baza danych RDS jest dostępna wyłącznie z wnętrza VPC — żaden zewnętrzny klient nie może połączyć się z nią bezpośrednio.

2.3 RAG Pipeline — przepływ danych

Poniżej przedstawiono szczegółowy przepływ przetwarzania każdej wiadomości użytkownika:

Krok	Moduł	Opis działania
1	MessageClassifier	LLM klasyfikuje wiadomość jako: rag (nowe zapytanie), followup (pytanie o wcześniej polecane miejsca) lub hybrid (nowe miejsca w kontekście historii)
2	MetadataExtractor	LLM wyodrębnia filtry z zapytania: dzielnica, kategoria, cena, godziny, liczba wyników. Generuje clean_query (zapytanie bez filtrów do embeddingu)
3	HyDE	Hypothetical Document Embedding — LLM generuje hipotetyczny opis miejsca dla lepszego semantic search

4	VectorStore	Wyszukiwanie wektorowe w bazie z predykatami metadanych. Post-filtrowanie godzin otwarcia w Pythonie (format JSONB nie obsługuje predykatów SQL)
5	Synthesizer	LLM generuje naturalną odpowiedź z kontekstem historii czatu (sliding window 5 wiadomości) i wynikami wyszukiwania
6	ChatRepository	Zapis wiadomości i poleconych miejsc do PostgreSQL. Kontekst umożliwia odpowiedź na followup bez ponownego wyszukiwania

2.4 Przepływ autentykacji

Autentykacja opiera się na AWS Cognito z przepływem USER_PASSWORD_AUTH. Użytkownik wpisuje email i hasło w aplikacji, Amplify SDK wywołuje Cognito i otrzymuje trzy tokeny: accessToken (ważny 1 godzinę), idToken (1 godzina) oraz refreshToken (30 dni). Tokeny są przechowywane w AsyncStorage urządzenia przez cognitoUserPoolsTokenProvider.

Każdy request do FastAPI zawiera accessToken w nagłówku Bearer. Backend weryfikuje podpis RS256 przy użyciu klucza publicznego JWKS i wyciąga z payloadu tokenu pole sub, które służy jako user_id do izolacji danych między użytkownikami. Gdy accessToken wygaśnie, Amplify automatycznie używa refreshToken do pobrania nowego.

Rejestracja wymaga weryfikacji adresu email kodem jednorazowym wysyłanym przez Cognito. Bez weryfikacji konto pozostaje nieaktywne i nie może być używane do logowania.

2.5 Zewnętrzne usługi i integracje

System integruje się z trzema zewnętrznymi usługami. Każda z nich wymaga osobnej konfiguracji i kluczy API:

Usługa	Zastosowanie	Konfiguracja
OpenAI API	Klasyfikacja zapytań, ekstrakcja metadanych, generowanie HyDE, synteza odpowiedzi — 3-4 wywołania per wiadomość użytkownika	OPENAI_API_KEY w .env backendu
AWS Cognito	Rejestracja, logowanie, wystawianie tokenów JWT, weryfikacja email. User Pool i App Client tworzone przez Terraform.	COGNITO_USER_POOL_ID i COGNITO_APP_CLIENT_ID w .env backendu i frontendu
Google Places API	Jednorazowe pobranie danych miejsc (nazwa, adres, godziny, atrybuty, koordynaty). Skrypt scripts/collect_places.py.	GOOGLE_API_KEY w .env potrzebny tylko przy ponownym zbieraniu danych

Wszystkie klucze API muszą być przechowywane wyłącznie w plikach .env lub jako sekrety w systemach CI/CD — nigdy w repozytorium kodu.

2.6 Backend — struktura i komponenty

Struktura katalogów

```
backend/
├── Dockerfile           # Konfiguracja kontenera produkcyjnego
├── requirements.txt      # Lista wymaganych bibliotek Python
├── docker-compose.yml   # Lokalny kontener TimescaleDB
├── app/
│   ├── main.py          # FastAPI app, CORS, middleware, security headers
│   ├── dependencies.py   # Weryfikacja JWT z AWS Cognito (JWKS + RS256)
│   ├── config/
│   │   └── settings.py  # Konfiguracja (Pydantic Settings)
│   ├── routers/
│   │   └── chat.py       # POST /chat/message, GET /chat/all-names, GET
│   │       /chat/search-history
│   │       ├── sessions.py # GET /sessions/, GET /sessions/{id}/messages, DELETE
│   │       └── places.py  # GET /places/all, GET /places/favourites, POST
│   │           /places/favourites/{id}/toggle
│   ├── services/
│   │   ├── chat_service.py # Orchestrator – główna logika RAG
│   │   ├── message_classifier.py
│   │   ├── metadata_extractor.py
│   │   ├── synthesizer.py
│   │   └── llm_factory.py  # Fabryka klienta OpenAI
│   ├── database/
│   │   ├── insert_vectors.py # Tworzenie tabel embeddings i places, generowanie
│   │   │       embeddingów, import danych z CSV
│   │   ├── vector_store.py  # Wyszukiwanie wektorowe (pgvector / TimescaleDB)
│   │   ├── chat_repository.py # Historia czatu, sesje
│   │   └── places_repository.py # Zapisane miejsca
│   ├── schemas/
│   │   ├── chat.py         # ChatRequest, ChatResponse, ClassificationResult,
│   │   │       SynthesizedResponse
│   │   ├── session.py      # SessionResponse, MessageResponse
│   │   └── places.py       # PlaceResponse, SavedPlaceResponse, ToggleFavouriteRequest
```

2.7 Frontend — struktura i komponenty

Struktura katalogów

```
frontend/
├── api/
│   ├── client.ts        # axios instance, interceptor JWT (fetchAuthSession)
│   ├── chat.ts          # sendMessage(text, sessionId)
│   ├── places.ts        # getAllPlaces, getFavouritePlaces, toggleFavourite
│   └── sessions.ts      # getSessions, getSessionMessages, deleteSession
├── app/
│   ├── (auth)/
│   │   ├── _layout.tsx  # Stack bez headera
│   │   ├── login.tsx    # Logowanie (USER_PASSWORD_AUTH)
│   │   └── register.tsx # Rejestracja + weryfikacja email kodem
│   ├── (tabs)/
│   │   ├── place/[name]/index.tsx # Szczegóły miejsca – dane z params lub placesStore
│   │   ├── _layout.tsx  # Tab navigator (5 zakładek)
│   │   ├── chat.tsx     # Czat RAG z historią sesji
│   │   ├── map.tsx      # Mapa z markerami i filtrowaniem
│   │   ├── saved.tsx    # Lista zapisanych miejsc (SectionList)
│   │   └── profile.tsx  # Profil, statystyki, wylogowanie
```

```

├── _layout.tsx      # Root layout (Amplify.configure)
├── index.tsx       # Punkt wejścia — przekierowanie auth
├── components/
│   ├── chat/       # Komponenty specyficzne dla ekranu czatu
│   ├── map/        # Komponenty specyficzne dla ekranu mapy
│   └── saved/       # Komponenty specyficzne dla ekranu ulubionych
├── constants/      # Stałe kolorów, kategorii
├── hooks/
│   ├── useGradientCycle.ts # Animowany gradient czatu
│   └── useFavourite.ts    # Toggle ulubionych z API sync
├── patches/
│   └── react-native-maps+1.20.1.patch # Fix dla Expo 54 (patrz sekcja 2.8)
├── store/
│   ├── authStore.ts      # isAuthenticated, user, setAuthenticated
│   ├── favouritesStore.ts # ids ulubionych, toggle, sync z API
│   └── placesStore.ts     # selectedPlace do nawigacji szczegółów
└── amplify-config.ts     # Cognito userPoolId, clientId, loginWith

```

Przegląd i główne biblioteki

Frontend to aplikacja React Native zbudowana na Expo SDK 54 z New Architecture (React Native 0.81). Używa Expo Router 6 do nawigacji plikowej — każdy plik w katalogu app/ automatycznie staje się ekranem lub layoutem, bez potrzeby ręcznego definiowania tras.

Komunikacja z backendem odbywa się przez axios z interceptorem JWT — przy każdym requencie automatycznie dołączany jest token dostępu z AWS Amplify. Jeśli sesja wygasła, Amplify automatycznie odświeża token przez refresh token przechowywany w AsyncStorage.

Stan aplikacji zarządzany jest przez Zustand — lekką bibliotekę bez boilerplate, z trzema store'ami: authStore (stan autentykacji), favouritesStore (ulubione miejsca z synchronizacją do API) i placesStore (wybrany kontekst miejsca przy nawigacji do szczegółów).

Wersje zależności są przypięte w package.json — instalacja przez npm install zawsze zainstaluje dokładnie te wersje.

Kluczowe zależności (package.json)

Pakiet	Wersja	Zastosowanie
expo	~54.0.33	Framework — SDK, narzędzia, zarządzanie native modules
expo-router	~6.0.23	Nawigacja plikowa, tab bar, deep linking
react-native	0.81.5	Core React Native (New Architecture / Fabric renderer)
react	19.1.0	React core
react-native-maps	1.20.1	Mapa z markerami (wymaga patcha — patrz sekcja 2.8)
react-native-map-clustering	^4.0.0	Automatyczne grupowanie bliskich markerów w klastry
react-native-reanimated	~4.1.1	Animacje natywne (Worklets, shared values)

react-native-gesture-handler	~2.28.0	Obsługa gestów dotykowych
react-native-safe-area-context	~5.6.0	Bezpieczne marginesy ekranu (notch, home bar)
react-native-screens	~4.16.0	Natywna nawigacja między ekranami
expo-linear-gradient	~15.0.8	Animowane tło czatu (LinearGradient)
expo-blur	~15.0.8	BlurView w headerach, kartach i modalach
expo-location	~19.0.8	Geolokalizacja użytkownika na mapie
expo-image	~3.0.11	Wydajne wyświetlanie obrazów z cache
expo-haptics	~15.0.8	Wibracje przy interakcjach
expo-font	~14.0.11	Ładowanie czcionek (DM Sans)
@expo-google-fonts/dm-sans	^0.4.2	Czcionka DM Sans
aws-amplify	^6.16.3	Autentykacja Cognito (USER_PASSWORD_AUTH)
@aws-amplify/react-native	^1.3.3	Natywne moduły Amplify (AsyncStorage)
@react-native-async-storage/async-storage	2.2.0	Przechowywanie tokenów JWT między sesjami
@react-native-community/netinfo	11.4.1	Wykrywanie stanu połączenia sieciowego
zustand	^5.0.12	Zarządzanie stanem globalnym (authStore, favouritesStore, placesStore)
axios	^1.14.0	HTTP klient z interceptorem JWT
react-native-uuid	^2.0.4	Generowanie UUID dla wiadomości czatu
react-native-markdown-display	^7.0.2	Renderowanie odpowiedzi asystenta w Markdown
patch-package	^8.0.1	Automatyczne stosowanie patchy po npm install
typescript	~5.9.2	Typowanie statyczne

Kompatybilność wersji

Projekt był testowany z Expo SDK 54 i React Native 0.81 z włączoną New Architecture (Fabric renderer). Przed aktualizacją którejkolwiek z poniższych bibliotek przeczytaj uwagi:

- Przy aktualizacji Expo SDK (np. z 54 na 55) sprawdź czy patch react-native-maps jest nadal wymagany — każda nowa wersja SDK może rozwiązać lub wprowadzić nową niekompatybilność. Changelog Expo dostępny jest na expo.dev/changelog.
- react-native-maps jest przypięty do dokładnej wersji 1.20.1 bez operatora ^ — wyższe wersje mogą nie wymagać patcha, ale mogą też wprowadzić nowe błędy z New Architecture. Przed aktualizacją przetestuj dokładnie działanie mapy.
- Biblioteki expo-* (blur, location, linear-gradient i inne) muszą być kompatybilne z wersją Expo SDK — operator ~ oznacza aktualizacje tylko w ramach patch releases. Przy zmianie SDK zaktualizuj je przez: `npx expo install --fix`

Widoki

Chat

Ekran czatu (`app/(tabs)/chat.tsx`) jest głównym widokiem aplikacji. Przy każdym wysłaniu wiadomości frontend wykonuje `POST /chat/message` z treścią wiadomości i `session_id`. Backend zwraca odpowiedź asystenta, listę poleconych miejsc (`PlaceResponse[]`), typ wiadomości oraz flagi `enough_context` i `session_id`.

Poleczone miejsca wyświetlane są w poziomym sliderze kart pod bańką asystenta — tylko gdy odpowiedź nie jest followupem (`is_followup: false`). Kliknięcie karty przekierowuje do ekranu szczegółów `place/[name]` z danymi przekazanymi przez `params` (JSON stringified).

Historia czatu ładowana jest przez `GET /sessions/` — ekran historii wyświetla sesje pogrupowane według daty (dziś, wczoraj, wcześniej) z pierwszą wiadomością jako tytułem. Po kliknięciu sesji ładowane są wiadomości przez `GET /sessions/{id}/messages`, a lista jest scrollowana do ostatniej wiadomości po wyrenderowaniu (opacity fade-in zapobiega widocznemu skokowi).

Tło czatu używa `useGradientCycle` — hooka który interpoluje między paletami kolorów przez `Animated.Value` z listenerem. Zmiana gradientu wyzwalana jest co 150px scroll listy wiadomości.

Mapa

Ekran mapy (`app/(tabs)/map.tsx`) używa `MapViewClustering` jako wrappera nad `react-native-maps`. Clustering automatycznie grupuje bliskie markery w okrągłe klastry z licznikiem.

Region mapy przechowywany jest w `useRef` (nie `useState`) żeby zmiany nie powodowały re-renderu. Markery kategorii (`CategoryPin`) renderowane są z `LinearGradient` wewnątrz `Marker`. Zaznaczony marker jest większy (38px vs 32px) z dodatkowym pierścieniem.

Po kliknięciu markera pojawia się poziomy slider kart (`PlaceCard`) zsynchronizowany z mapą — przewijanie slidera animuje mapę do kolejnego miejsca, a kliknięcie markera scrolluje slider do właściwej karty. Synchronizacja kontrolowana jest przez `handleViewable` i `suppressViewable` ref, który zapobiega pętli zdarzeń.

Dostępne filtry: kategoria główna (`saved/restauracje/kawiarnie/bary/kultura`), podkategoria, poziom cen, dzielnica oraz wyszukiwanie tekstowe po nazwie. `FilterResultPopup` pokazuje liczbę wyników po zastosowaniu filtra.

Ulubione

Ekran ulubionych (`app/(tabs)/saved.tsx`) wyświetla miejsca zapisane przez użytkownika jako `SectionList` posortowany według kategorii głównej. Każde miejsce można usunąć z ulubionych przez kliknięcie ikony serduszka.

Stan ulubionych jest synchronizowany przez `favouritesStore` z `Zustand` — przy starcie aplikacji store ładuje listę ID ulubionych przez `GET /places/favourite-names`, a każdy toggle przez `POST /places/favourites/{id}/toggle` aktualizuje jednocześnie bazę danych i lokalny store. Dzięki temu serduszka na ekranie mapy i czatu są zawsze aktualne bez dodatkowych requestów.

Szczegóły miejsca

Widok dostępny po kliknięciu karty miejsca z czatu lub mapy. Zawiera pełne dane miejsca: adres, godziny otwarcia, ocenę, tagi (np. `serves_coffee`, `outdoor_seating`), link do Google Maps oraz link do strony i menu jeśli dostępny. Przycisk serduszka pozwala dodać lub usunąć miejsce z ulubionych.

Profil

Ekran profilu (`app/(tabs)/profile.tsx`) wyświetla adres email użytkownika pobrany z tokenu JWT. Przycisk wylogowania wywołuje `signOut` z `AWS Amplify`, który unieważnia tokeny `Cognito` i czyści `AsyncStorage`, a następnie przekierowuje na ekran logowania przez `authStore`.

2.8 Patch biblioteki react-native-maps

react-native-maps w wersji 1.20.1 zawiera błąd powodujący crash aplikacji w Expo SDK 54 z New Architecture. react-native-maps w wersji 1.20.1 zawiera błąd powodujący crash aplikacji w Expo SDK 54 z New Architecture. Patch jest wymagany i aplikowany automatycznie przy każdym `npm install`.

Problem

Biblioteka `react-native-maps 1.20.1` zawiera błąd powodujący crash aplikacji w `Expo SDK 54` z `New Architecture`. Objawia się w trybie deweloperskim przy wykonaniu `zoom in` / `zoom out` na mapie, a w `Xcode Console` widoczny jest następujący błąd:

```
NSInvalidArgumentException *** -[__NSArrayM insertObject:atIndex:]: object cannot be nil
```

Przyczyną jest niezgodność wewnętrznego komponentu biblioteki z `Fabric renderer` (`React Native 0.81` / `Expo SDK 54`). Patch jest wymagany — bez niego mapa nie działa poprawnie.

Plik patcha znajduje się w `patches/react-native-maps+1.20.1.patch` i jest stosowany automatycznie przy każdym `npm install` dzięki skryptowi `postinstall` w `package.json`:

```
"postinstall": "patch-package"
```

3. Dane — źródła, przetwarzanie i struktura wektorowa

3.1 Źródło danych — Google Places API

Dane miejsc pochodzą z Google Places API i są pobierane jednorazowo przez skrypt `scripts/collect_places.py`. Skrypt wymaga klucza Google Places API w pliku `.env` — można go uzyskać bezpłatnie w Google Cloud Console z miesięcznym kredytem 300 USD.

Google kategoryzuje pola na trzy warstwy cenowe: Basic (nazwa, adres, koordynaty — 0,017 USD/1000 zapytań), Contact (telefon, strona, godziny otwarcia) oraz Atmosphere (editorial_summary, atrybuty opisowe — 0,032 USD/1000). Przy jednorazowym pobraniu miejsc całkowity koszt mieści się w darmowym limicie.

Część pól (`menu_url`, `district`) nie była dostępna bezpośrednio przez API — brakujące dane zostały uzupełnione z pomocą modelu językowego z dostępem do internetu oraz własnych funkcji. Warto pamiętać, że dane po pobraniu wymagają preprocessingu (patrz `preprocessing.ipynb`) — można tam uzupełnić brakujące pola, usunąć te które nie są potrzebne, a także odfiltrować miejsca które nie pasują do charakteru aplikacji, np. sklepy sieciowe (Żabka, Biedronka, Putka), stacje benzynowe czy inne przypadkowo pobrane lokalizacje. Mimo tego nie wszystkie pola są gwarantowane — niektóre miejsca mogą nie mieć uzupełnionego `menu_url` lub `website`.

Pola zbierane przez skrypt (Google Places API)

Pole	Opis
<code>place_id</code>	Id miejsca
<code>name</code>	Nazwa miejsca
<code>address</code>	Pełny adres
<code>lat, lon</code>	Współrzędne geograficzne
<code>adress_context</code>	Dodatkowy kontekst adresu
<code>rating</code>	Średnia ocena (0–5)
<code>user_rating_count</code>	Liczba ocen
<code>price_range start, price_range_end</code>	Widelki cenowe
<code>price_level</code>	Poziom cen
<code>primary_type</code>	Główny typ miejsca według Google
<code>types</code>	Lista wszystkich typów przypisanych przez Google
<code>opening_hours</code>	Godziny otwarcia (tablica tekstów)
<code>all_reviews</code>	Opinie użytkowników (ostatnie 3)
<code>editorial_summary</code>	Redakcyjny opis miejsca (Google)
<code>maps_url</code>	Link do Google Maps
<code>website</code>	Adres strony internetowej

serves_vegetarian, serves_coffee, serves_beer, serves_wine, serves_cocktails, serves_breakfast, serves_lunch, serves_dinner, serves_dessert, outdoor_seating, good_for_groups, reservable, takeout, dine_in, menu_for_children	Atrybuty boolowskie opisujące ofertę i udogodnienia miejsca
--	---

Pola utworzone samodzielnie

Pole	Opis
district	Dzielnica Warszawy
menu_url	Link do menu (jeśli dostępny)
main_category	Kategoria główna (np. Gastronomia, Kawa i Słodycze, Życie Nocne)
sub_category	Podkategoria (np. Kuchnia Polska, Kawiarnia, Bar)

Miejsca końcowo zostały podzielone na pięć głównych kategorii:

- **Gastronomia** — Kuchnia Polska, Kuchnia Włoska, Kuchnia Azjatycka, Kuchnia Indyjska, Kebab & Bliskowschodnia, Burgery & Amerykańska, Fast Food & Przekąski, Kuchnia Europejska & Śródziemnomorska, Kuchnia Latinoamerykańska, Kuchnie Świata (Inne), Owoce Morza & Ryby, Kuchnia Roślinna, Steki & Grill, Restauracja
- **Kawa i Słodycze** — Kawiarnia, Piekarnia & Cukiernia, Lody & Zimne Desery
- **Życie Nocne** — Bar, Klub
- **Kultura & Rozrywka** — Muzeum & Galeria, Atrakcje & Zabytki, Edukacja & Nauka, Rozrywka Aktywna
- **Natura & Rekreacja** — Park & Ogród, ZOO & Akwarium

3.2 Pipeline przetwarzania danych

Po pobraniu i preprocessingu danych CSV uruchamiasz skrypt który tworzy tabele w bazie, generuje embeddingi wektorowe przez OpenAI API i zapisuje wszystko do bazy. Skrypt wykonuje się jednorazowo — kolejne uruchomienia pomijają już istniejące miejsca przez ON CONFLICT DO NOTHING.

Przed wykonaniem poniższej instrukcji należy uruchomić kontener z bazą danych oraz upewnić się, że instancja PostgreSQL jest dostępna i gotowa do przyjmowania połączeń (punkt 7.3).

```
# Upewnij się że .env zawiera OPENAI_API_KEY i POSTGRES_CONNECTION_STRING

cd backend/app
python -m database.insert_vectors # Przy 500+ miejscach: 5-10 minut, koszt ~0.01 USD
```

3.3 Struktura wektorowa i wyszukiwanie semantyczne

Każde miejsce jest reprezentowane przez wektor 1536-wymiarowy wygenerowany modelem text-embedding-3-small. Wektory przechowywane są w tabeli embeddings z indeksem HNSW (pgvector) — umożliwiającym szybkie wyszukiwanie kosinusowe.

Pipeline wyszukiwania: zapytanie użytkownika → clean_query (bez filtrów metadanych) → HyDE (hipotetyczny dokument) → embedding → szukanie <=> w pgvector z predykatami metadanych (district, price_level, main_category) → top N wyników do Synthesizera.

4. Schemat bazy danych

4.1 Tabele

System używa jednej instancji PostgreSQL z rozszerzeniem pgvector do przechowywania wszystkich danych. Lokalnie działa na TimescaleDB (Docker), na produkcji na AWS RDS PostgreSQL 15.

Tabela	Typ	Opis
embeddings	pgvector (PostgreSQL)	Wektory miejsc z pełnymi metadanymi (JSONB)
places	PostgreSQL	Wszystkie dane miejsc (Google Place ID jako PK). Tabela tworzona i zasilana danymi z pliku warsaw_places.csv
chat_sessions	PostgreSQL	Sesje użytkowników (id, user_id, created_at)
chat_messages	PostgreSQL	Wiadomości z historii czatu (session_id, role, content, message_type, recommended_places[], created_at)
saved_places	PostgreSQL	Ulubione miejsca użytkowników — relacja user ↔ place (FK do places, UNIQUE user+place_id)

4.2 Tabela embeddings — kluczowe pola

Kolumna	Typ	Opis
id	UUID	Unikalny identyfikator
metadata	JSONB	Wszystkie atrybuty miejsca: name, address, district, rating, price_level, godziny, booleany (serves_coffee, outdoor_seating, ...)
contents	TEXT	Treść tekstowa do embeddingu (nazwa + opis + opinie)
embedding	VECTOR(1536)	Wektor semantyczny (text-embedding-3-small)

4.3 Tabela saved_places — schemat kolumn

Kolumna	Typ	Opis
id	UUID (PK)	Automatycznie generowany gen_random_uuid()
user_id	TEXT	Sub claim z tokenu JWT Cognito
place_id	TEXT (FK)	Klucz obcy do places (Google Place ID, ChIJ...)
session_id	TEXT	Sesja w której miejsce zostało polecone
is_favourite	BOOLEAN	Czy miejsce jest w ulubionych (toggle)
created_at	TIMESTAMPTZ	Czas dodania do ulubionych (UTC)

4.4 Tabela chat_sessions

Kolumna	Typ	Opis
id	UUID (PK)	gen_random_uuid() — unikalny identyfikator sesji
user_id	TEXT	Sub claim z tokenu JWT — właściciel sesji
created_at	TIMESTAMPTZ	Czas utworzenia sesji (UTC)

4.5 Tabela chat_messages

Kolumna	Typ	Opis
id	UUID (PK)	gen_random_uuid()
session_id	UUID (FK)	Klucz obcy do chat_sessions
role	TEXT	'user' lub 'assistant'
content	TEXT	Treść wiadomości
message_type	TEXT	Typ: 'rag', 'followup', 'hybrid'
recommended_places	JSONB[]	Lista id poleconych miejsc dołączona do odpowiedzi asystenta
created_at	TIMESTAMPTZ	Czas wysłania wiadomości (UTC)

Schemat bazy danych zarządzany jest ręcznie, bez dedykowanego narzędzia migracji — podejście świadomie uproszczone na potrzeby projektu w niewielkiej grupie, gdzie zmiany schematu zdarzają się rzadko. Odpowiada za to kilka skryptów: `database/insert_vectors.py` wczytuje dane z pliku `warsaw_places.csv` i tworzy tabele `embeddings` i `places`, natomiast `database/chat_repository.py` i `database/places_repository.py` tworzą tabele `chat_sessions`, `chat_messages` i `saved_places` przy starcie aplikacji (`CREATE TABLE IF NOT EXISTS`).

Przy przejęciu projektu przez zespół zalecane jest wprowadzenie Alembic — umożliwia śledzenie historii zmian schematu i deterministyczny rollback bez potrzeby sięgania po snapshoty RDS.

Rollback schematu

Przy braku Alembic rollback wymaga ręcznej operacji DROP lub przywrócenia snapshotu RDS. Snapshoty tworzone automatycznie co dobę z retention 7 dni (sekcja 11.13) — point-in-time recovery dostępne przez AWS Console.

Przebudowanie indeksu HNSW po masowym imporcie

Przy dodaniu ponad 10% nowych wierszy zalecane przebudowanie indeksu:

```
REINDEX INDEX CONCURRENTLY idx_embeddings_hnsw;
```

Schemat bazy

Moduł wyszukiwania AI

embeddings	
id	UUID
metadata	JSONB
contents	TEXT
embedding	VECTOR(1536)
idx_embeddings_hnsw	HNSW

1:N

places	
place_id	TEXT (PK)
name, address, district, rating...	

FK

Historia użytkownika

chat_sessions	
id	UUID (PK)
user_id	TEXT
created_at	TIMESTAMPTZ

1:N

chat_messages	
id	UUID (PK)
session_id	UUID (FK)
role	TEXT
content	TEXT
message_type	TEXT
recommended_places	JSONB[]
created_at	TIMESTAMPTZ

saved_places	
id	UUID (PK)
user_id	TEXT
place_id	TEXT (FK)
session_id	TEXT
is_favourite	BOOLEAN

---- FK (klucz obcy)

→ relacja 1:N

5. API — Endpointy

5.1 Autoryzacja

Wszystkie endpointy (oprócz /health) wymagają nagłówka Authorization z tokenem JWT z AWS Cognito:

```
Authorization: Bearer <access_token>
```

Token jest weryfikowany przez pobieranie publicznych kluczy JWKS z endpointu Cognito i walidację podpisu RS256. Middleware sprawdza również scheme — akceptowane jest wyłącznie Bearer z wielką literą B.

5.2 Chat

Metoda	Endpoint	Opis
POST	/chat/message	Wysyła wiadomość użytkownika. Uruchamia pełny RAG pipeline (klasyfikacja, ekstrakcja, HyDE, vector search, synteza). Zwraca: answer, type, session_id, places[], enough_context.
GET	/chat/all-names	System zwraca listę unikalnych nazw wszystkich miejsc z bazy. Jest ona wykorzystywana przez frontend do dynamicznego wyszukiwania konkretnych miejsc na podstawie wpisanego przez użytkownika tekstu (inputu.)
GET	/chat/search-history	Przeszukuje historię wiadomości użytkownika po frazie (query param: q). Zwraca listę sesji zawierających szukaną frazę.

5.3 Miejsca

Metoda	Endpoint	Opis
GET	/places/all	Zwraca wszystkie miejsca z podstawowymi polami (name, lat, lon, main_category, sub_category, rating, district). Używane przez mapę.
GET	/places/favourites	Zwraca miejsca zapisane przez użytkownika z pełną strukturą.
POST	/places/favourites/{place_id}/toggle	Przełącza status ulubionego. Zwraca { is_favourite: bool }.
GET	/places/favourite-names	Zwraca listę nazw wszystkich ulubionych. Używane przy inicjalizacji FavouritesStore.

5.4 Sesje

Metoda	Endpoint	Opis
GET	/sessions/	Zwraca listę wszystkich sesji użytkownika posortowanych od najnowszej. Każda sesja zawiera: id, created_at, first_message, message_count.
GET	/sessions/{session_id}/messages	Zwraca pełną historię wiadomości dla podanej sesji w kolejności chronologicznej.
DELETE	/sessions/{session_id}	Trwale usuwa sesję wraz ze wszystkimi jej wiadomościami. Operacja nieodwracalna.

6. Obsługa błędów i fallbacki

6.1 Backend — kody odpowiedzi i zachowanie

Backend stosuje warstwową obsługę błędów. Błędy walidacji (Pydantic) zwracają 422 z opisem pola. Błędy autoryzacji zwracają 401 lub 403. Wyjątki serwera przechwytywane są przez globalny exception handler i logowane do `app.log` — klient nigdy nie otrzymuje stack trace ani wewnętrznych szczegółów implementacji.

Logowanie skonfigurowane jest na poziomie **INFO** z dwoma handlerami działającymi równolegle: `FileHandler` zapisuje wszystkie zdarzenia trwale do `app.log`, natomiast `StreamHandler` wypisuje je na stdout — na środowisku AWS przekłada się to na CloudWatch. Błędy 500 i 503 logowane są na poziomie **ERROR**, zdarzenia autoryzacyjne (np. `AUTH_FAILURE`) na poziomie **WARNING**, a standardowe requesty na poziomie **INFO**. Format każdego wpisu zawiera timestamp, poziom, nazwę logger'a oraz treść wiadomości. Logi **DEBUG** nie są zbierane na produkcji.

Scenariusz błędu	Kod HTTP	Zachowanie systemu
Brak lub nieważny token JWT	401	{'detail': 'Not authenticated'} — brak stack trace, zdarzenie logowane jako <code>AUTH_FAILURE</code>
Token z innego user pool / zły podpis	401	Weryfikacja JWKS odrzuca podpis RS256 — 401
Dostęp do cudzego zasobu (IDOR)	403	user_id z tokenu nie pasuje do zasobu — 403 Forbidden
Nieprawidłowy JSON w body requestu	422	Pydantic zwraca listę błędów walidacji z nazwami pól
Nieoczekiwany wyjątek serwera	500	{'detail': 'Internal server error'} — stack trace tylko w <code>app.log</code> , klient dostaje generyczny komunikat
Niedostępne OpenAI API (timeout 30 s)	503	{'detail': 'AI service unavailable'} — błąd logowany, klient otrzymuje czytelny komunikat

Niedostępna baza danych PostgreSQL	503	psycopg2 connection error — 503, wpis w app.log
Nieistniejący zasób	404	{'detail': 'Session not found'} lub analogiczny
SQL injection w parametrach	422	Pydantic odrzuca input lub psycopg2 parametryzuje zapytanie — brak wykonania
Ekstremalne wartości liczbowe	422	Pydantic validator — odrzucone przed trafieniem do bazy danych

6.2 Frontend — obsługa błędów po stronie klienta

Axios interceptor przechwytuje wszystkie błędy HTTP. Tokeny odświeżane są automatycznie przez Amplify — użytkownik nie widzi błędu wygaśnięcia sesji. Aplikacja nie crashuje przy błędach sieciowych — wyświetla komunikat inline lub toast.

Scenariusz	Typ błędu	Zachowanie aplikacji
Brak połączenia sieciowego	Network Error	Komunikat inline — aplikacja nie crashuje, dane lokalne dostępne
Timeout odpowiedzi backendu (> 30 s)	408 / axios timeout	Komunikat 'Spróbuj ponownie' — wiadomość nie trafia do historii sesji
Wygaśnięcie accessToken	401 → auto-refresh	Amplify transparentnie pobiera nowy token — request powtarzany bez ingerencji użytkownika
Wygaśnięcie refreshToken (> 30 dni)	401 → logout	authStore.setAuthenticated(false), redirect na ekran logowania
Pusta lista wyników wyszukiwania	200 + []	Stan 'Nie znaleziono rozmów' zamiast pustego widoku
Brak zgody na geolokalizację	Odmowa uprawnień	Mapa wyświetlana bez markera użytkownika — bez komunikatu błędu
Mapa w Expo Go (oczekiwany crash)	N/A	Dokumentowane — do testowania mapy wymagany dev build (sekcja 12.2)

6.3 Limity zewnętrznych usług

Usługa	Limit	Zachowanie po przekroczeniu
OpenAI API	TPM / RPM wg planu	HTTP 429 → backend zwraca 503, zdarzenie logowane
Google Places API	Jednorazowy scraping — brak live calls	Nie dotyczy — dane załadowane statycznie do bazy danych
AWS Cognito	Bez limitu przy małej skali	Throttling przy ekstremalnym ruchu — w praktyce niekrytyczne dla MVP

7. Diagramy przepływu

7.1 Przepływ wiadomości czatu

```
[Uzytkownik] --> Wiadomosc
|
v
[ChatService] --> pobierz historie z DB
|
v
[MessageClassifier] --> rag | followup | hybrid
|                               |
v (rag/hybrid)                 v (followup)
[MetadataExtractor]           [pobierz kontekst z DB]
|
v
[HyDE] --> rozszerz clean_query
|
v
[VectorStore] --> semantic search + predykaty metadanych
|
v
[Synthesizer] --> odpowiedz + recommended_places
|
v
[ChatRepository] --> zapis do DB
|
v
[Uzytkownik] <-- ChatResponse (answer + places[])
```

7.2 Przepływ autentykacji

1. Użytkownik wpisuje email i hasło w aplikacji
2. Amplify SDK wywołuje Cognito z USER_PASSWORD_AUTH flow
3. Cognito zwraca accessToken (1h), idToken (1h), refreshToken (30 dni)
4. Tokeny zapisywane w AsyncStorage przez cognitoUserPoolsTokenProvider
5. Każdy request do FastAPI zawiera Bearer token w nagłówku (axios interceptor)
6. Backend pobiera JWKS z Cognito (cache @lru_cache) i weryfikuje podpis RS256
7. Z payloadu tokenu wyciągany jest user_id (sub claim) do izolacji danych
8. Gdy token wygasa, Amplify automatycznie odświeża go przez refresh token

8. Instrukcja uruchomienia — lokalnie

8.1 Wymagania wstępne

Narzędzie	Wersja minimalna	Zastosowanie
Python	3.11+	Backend FastAPI
Node.js	18+	Frontend React Native / Expo
Docker Desktop	4.0+	Kontener TimescaleDB
Terraform	1.5+	Provisioning infrastruktury AWS (patrz rozdział 11)
AWS CLI	2.0+	Dostęp do AWS (skonfigurowane credentials)
Expo Go	Aktualna	Testowanie aplikacji mobilnej bez kompilacji

8.2 Konfiguracja środowiska

Stwórz i uzupełnij plik `.env` w katalogu `backend/app/` na podstawie poniższego szablonu. Klucz `OPENAI_API_KEY` uzyskasz na platformie `platform.openai.com/api-keys` — wymagane jest konto z aktywnym planem płatnym. Zmienne Cognito będą dostępne dopiero po wykonaniu kroku 10.3 (Provisioning AWS Terraform) — wypełnij je po uruchomieniu Terraform:

```
# backend/app/.env
OPENAI_API_KEY=sk-...
POSTGRES_CONNECTION_STRING=postgresql://postgres:password@localhost:5432/warsaw_places
GOOGLE_MAPS_API_KEY=AIza..

# Uzupełnij po wykonaniu terraform apply -target=module.cognito
COGNITO_USER_POOL_ID=eu-north-1_XXXXXXXXXX
COGNITO_APP_CLIENT_ID=XXXXXXXXXXXXXXXXXXXXXXXXXXXX
COGNITO_REGION=eu-north-1

ENVIRONMENT=dev
```

Plik `frontend/.env` w katalogu `frontend/` tworzysz analogicznie:

```
# frontend/.env
API_URL=http://TWOJE_IP:8000

# Uzupełnij po wykonaniu terraform apply -target=module.cognito
COGNITO_USER_POOL_ID=eu-north-1_XXXXXXXXXX
COGNITO_APP_CLIENT_ID=XXXXXXXXXXXXXXXXXXXXXXXXXXXX
COGNITO_REGION=eu-north-1
```

Oba pliki `.env` są wykluczone z repozytorium przez `.gitignore`. Wzorzec z placeholderami dostępny jest w plikach `.env.example` w obu katalogach.

8.3 Uruchomienie bazy danych

1. Upewnij się że Docker Desktop jest uruchomiony
2. Przejdź do katalogu głównego projektu
3. Uruchom kontener TimescaleDB:

```
docker compose up -d
```

4. Sprawdź czy kontener działa:

```
docker ps
# Kontener timescaledb powinien mieć status Up
```

5. Zainstaluj wymagane rozszerzenia PostgreSQL:

```
docker exec -it timescaledb psql -U postgres -d warsaw_places

-- Zainstaluj rozszerzenia (wykonaj raz)
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS timescaledb;

-- Zweryfikuj instalacje
\dx
```

8.4 Pobranie i załadowanie danych

Krok 1 — Pobranie danych z Google Places API

```
cd backend
source .venv/bin/activate      # macOS/Linux
pip install -r app/requirements.txt

# Pobierz dane (wymaga GOOGLE_API_KEY w .env lub jako argument)
python scripts/collect_places.py
# Skrypt tworzy plik data/places.csv
```

Krok 2 — Wgranie danych do bazy i generowanie embeddingów

```
# Upewnij się że kontener TimescaleDB działa (docker ps)
# Upewnij się że .env zawiera OPENAI_API_KEY i POSTGRES_CONNECTION_STRING

cd backend/app
python -m database.insert_vectors
```

8.5 Provisioning AWS (Terraform)

Infrastruktura AWS — w tym Cognito — jest tworzona przez Terraform opisany w rozdziale 10. Lokalnie na tym etapie wystarczy uruchomić tylko moduł Cognito:

```
terraform apply -target=module.cognito
```

Pełna instrukcja konfiguracji i uruchomienia Terraform znajduje się w sekcji 10.3. Po wykonaniu polecenia skopiuj wartości outputów do obu plików .env:

```
terraform output cognito_user_pool_id    # -> COGNITO_USER_POOL_ID
terraform output cognito_app_client_id    # -> COGNITO_APP_CLIENT_ID
```

8.6 Uruchomienie backendu

1. Upewnij się że wirtualne środowisko jest aktywne
2. Przejdź do katalogu backend/app/
3. Uruchom serwer FastAPI:

```
uvicorn main:app --reload --port 8000 --host 0.0.0.0
```

4. Zweryfikuj działanie:

```
curl http://localhost:8000/health  
# Oczekiwana odpowiedź: {"status": "ok"}
```

5. Dokumentacja API dostępna pod adresem: <http://localhost:8000/docs>

8.7 Uruchomienie frontendu

Przed uruchomieniem upewnij się, że frontend/.env zawiera prawidłowe wartości

- API_URL,
- GOOGLE_MAPS_API_KEY,
- COGNITO_USER_POOL_ID
- COGNITO_APP_CLIENT_ID.

Klucz Google Maps API — wymagania per platforma

Aby mapy działały, potrzebny jest klucz Google Maps API. Uzyskasz go w [Google Cloud Console](#):

1. Utwórz projekt lub wybierz istniejący
2. Przejdź do **APIs & Services → Library** i włącz odpowiednie SDK:
 - a. **Android**: Maps SDK for Android — wymagany obowiązkowo
 - b. **iOS**: Maps SDK for iOS — wymagany tylko jeśli używasz Google Maps jako providera na iOS; jeśli używasz domyślnego Apple Maps, klucz nie jest potrzebny
 - c. Przejdź do **APIs & Services → Credentials → Create Credentials → API Key**
3. Skopiuj klucz i wklej go do frontend/ .env jako GOOGLE_MAPS_API_KEY

Przy testowaniu przez Expo Go dodatkowa konfiguracja nie jest wymagana. Klucz API jest natomiast niezbędny przy budowaniu binarki do App Store lub Google Play.

Następnie:

1. Przejdź do katalogu frontend/
2. Zainstaluj zależności — patch zostanie zastosowany automatycznie:

```
npm install
```

3. Uruchom Expo:

```
npx expo start
```

4. Zeskanuj kod QR aplikacją Expo Go lub naciśnij *i* dla symulatora iOS / *a* dla Android.

Telefon i komputer muszą być w tej samej sieci Wi-Fi. Jeśli połączenie nie działa, użyj trybu tunnel: `npx expo start --tunnel` (wymaga konta Expo i biblioteki `@expo/ngrok`).

9. Konfiguracja i parametry

9.1 Parametry modeli AI

Parametr	Wartość	Opis
Główny model LLM	gpt-5.4-mini	Klasyfikacja, ekstrakcja, synteza
Model embeddingów	text-embedding-3-small	Wektory 1536 wymiarów
Limit wyników (domyślny)	5	Liczba polecanych miejsc
Sliding window historii	5 wiadomości	Kontekst dla LLM w Synthesizerze

9.2 Zmienne środowiskowe

Zmienna	Wymagana	Katalog	Opis
OPENAI_API_KEY	Tak	backend/	Klucz API OpenAI — uzyskaj na platform.openai.com
POSTGRES_CONNECTION_STRING	Tak	backend/	Connection string do dockerowej bazy PostgreSQL / TimescaleDB
ENVIRONMENT	Nie	backend/	Ustawienie 'prod' wyłącza /docs i /redoc na produkcji
COGNITO_USER_POOL_ID	Tak	backend/ + frontend/	ID puli użytkowników Cognito — dostępne po terraform apply
COGNITO_APP_CLIENT_ID	Tak	backend/ + frontend/	ID klienta aplikacji Cognito — dostępne po terraform apply
COGNITO_REGION	Tak	backend/ + frontend/	Region AWS, np. eu-north-1
API_URL	Tak	frontend/	Adres backendu: lokalnie IP komputera, produkcyjnie domena
GOOGLE_MAPS_API_KEY	Tak	frontend/	Klucz API Google Maps

10. Znane ograniczenia i uwagi

10.1 Ograniczenia techniczne

- Mapa (react-native-maps) crashuje w Expo Go nawet po zastosowaniu patcha — wymagany development build (szczegóły w sekcji 11.2)
- Backend musi być dostępny w tej samej sieci lokalnej co urządzenie mobilne podczas developmentu
- ECS Fargate działa w publicznych subnetach (brak NAT Gateway = niższy koszt) — bezpieczeństwo zapewniają security groups (patrz sekcja 11.4)

10.2 Koszty API

- Każda wiadomość użytkownika generuje 3-4 wywołania gpt-5.4-mini (klasyfikacja, ekstrakcja, synteza) + 1 wywołanie embeddings
- Szacunkowy koszt: ~0.01-0.05 USD per wiadomość (zależnie od długości kontekstu)
- Ładowanie danych: jednorazowy koszt kilkudziesięciu USD

10.3 Bezpieczeństwo — odesłanie

Szczegółowe omówienie bezpieczeństwa API i mobilnego — w tym wyniki testów OWASP Top 10:2025, OWASP Mobile Top 10:2024 i MASVS — znajduje się w rozdziale 13.

11. Testy manualne — potwierdzenie działania funkcji

Wszystkie kluczowe funkcje aplikacji zostały przetestowane manualnie na fizycznym urządzeniu iOS (iPhone) przy użyciu development build skompilowanego przez Xcode. Testy przeprowadzono w środowisku lokalnym, jak i produkcyjnym — aplikacja komunikowała się z backendem pod adresem <https://spotguide.me>, bazą danych AWS RDS PostgreSQL oraz usługami AWS Cognito i OpenAI.

Autentykacja

Funkcja	Status	Uwagi
Rejestracja nowego konta	✓	Email weryfikacyjny dociera w < 30 s, kod aktywuje konto poprawnie
Logowanie (poprawne dane)	✓	Tokens JWT pobierane, przekierowanie na ekran czatu
Logowanie (błędne hasło)	✓	Komunikat błędu bez ujawnienia szczegółów technicznych
Automatyczne odświeżanie tokenu	✓	Sesja trwa ponad 1 h bez wylogowania — Amplify odświeża transparentnie
Wylogowanie	✓	Tokens unieważnione, AsyncStorage wyczyszczony, redirect na ekran logowania

Czat — RAG Pipeline

Funkcja	Status	Uwagi
Zapytanie semantyczne (nowe miejsca)	✓	Odpowiedź asystenta + slider z kartami miejsc w < 10 s
Followup w tej samej sesji	✓	Odpowiedź kontekstowa, is_followup: true, bez ponownego wyszukiwania
Historia sesji — ładowanie i grupowanie	✓	Sesje pogrupowane wg daty: dziś / wczoraj / wcześniej
Historia sesji — wczytanie wiadomości	✓	Scroll do ostatniej wiadomości, fade-in bez widocznego skoku
Wyszukiwanie w historii czatu	✓	Debounced input, wyniki pg_trgm, stan 'Nie znaleziono rozmów'
Usunięcie sesji	✓	Sesja usuwana z listy bez przeładowania całego ekranu
Renderowanie Markdown w odpowiedzi	✓	Długie odpowiedzi renderowane poprawnie przez react-native-markdown-display

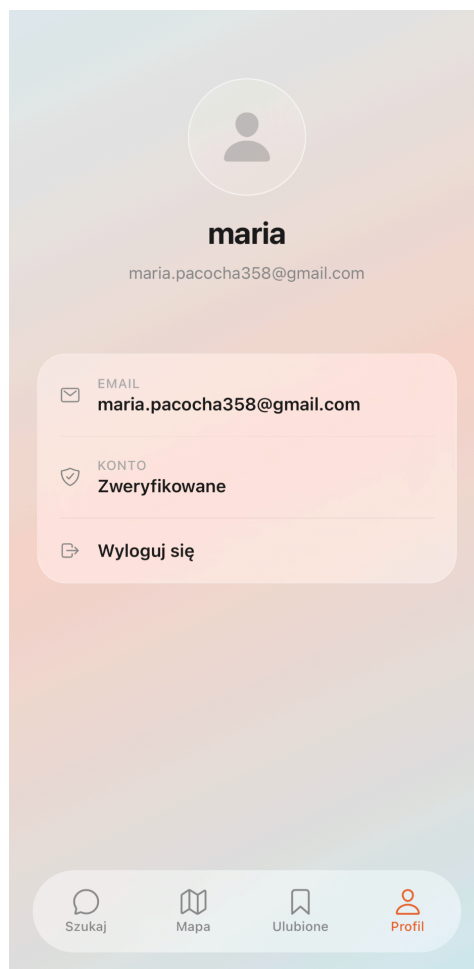
Ulubione, szczegóły miejsca, profil

Funkcja	Status	Uwagi
Dodanie/usunięcie z ulubionych (toggle)	✓	Sync z API i Zustand store — serduszka aktualne na wszystkich ekranach
Ekran szczegółów — dane	✓	Adres, godziny, ocena, tagi, link Google Maps / menu
Profil — email i statystyki	✓	Email z tokenu JWT

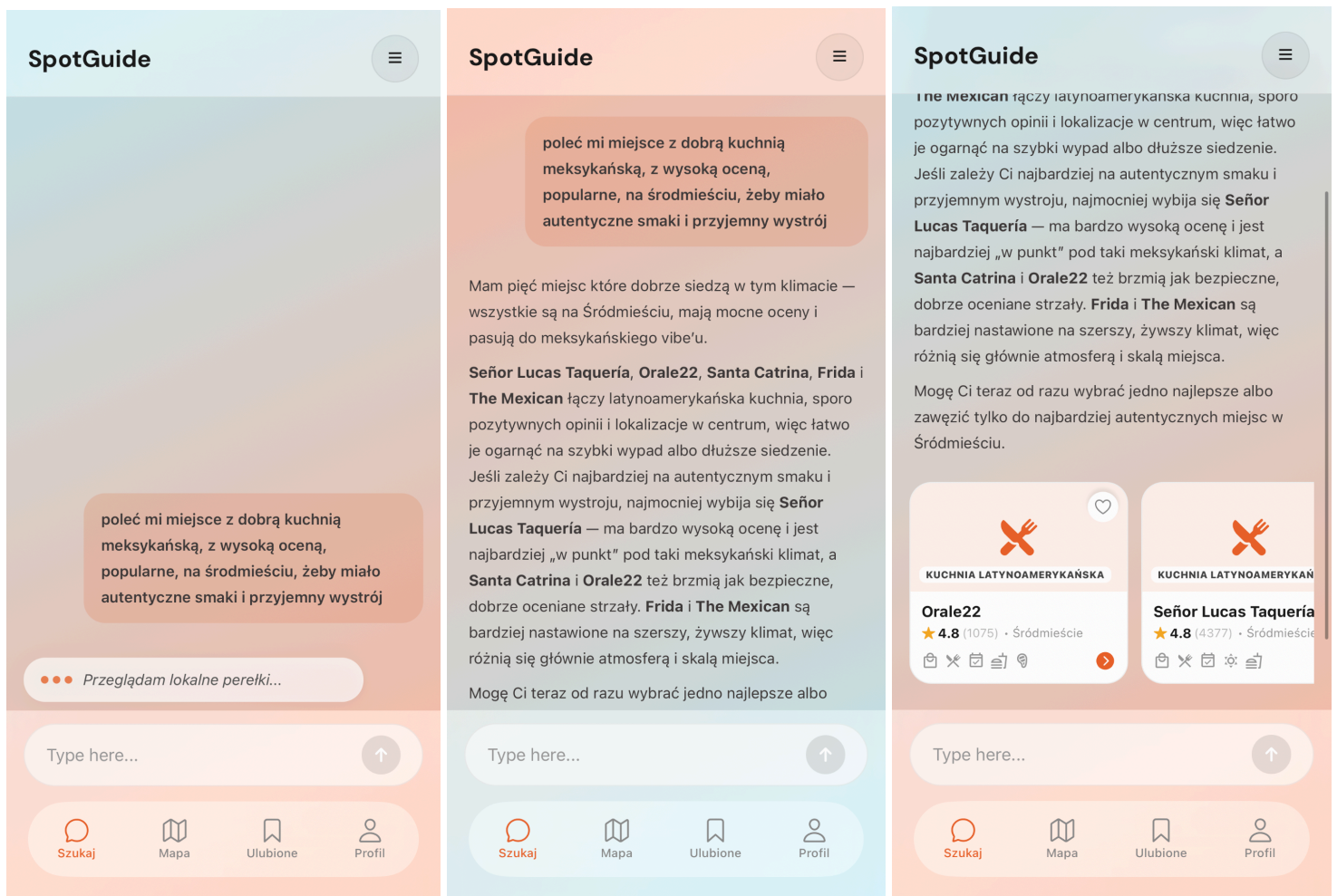
Mapa

Funkcja	Status	Uwagi
Wyświetlenie mapy z markerami	✓	Patch react-native-maps zastosowany automatycznie — brak crasha przy zoom in/out
Clustering markerów	✓	Bliskie markery grupowane w klastry z licznikiem
Kliknięcie markera → slider kart	✓	Synchronizacja mapa ↔ slider bez pętli zdarzeń
Filtrowanie (kategoria, cena, dzielnica)	✓	FilterResultPopup pokazuje liczbę wyników po zastosowaniu filtra
Geolokalizacja użytkownika	✓	Pulsujący marker po udzieleniu zgody — brak błędu przy odmowie

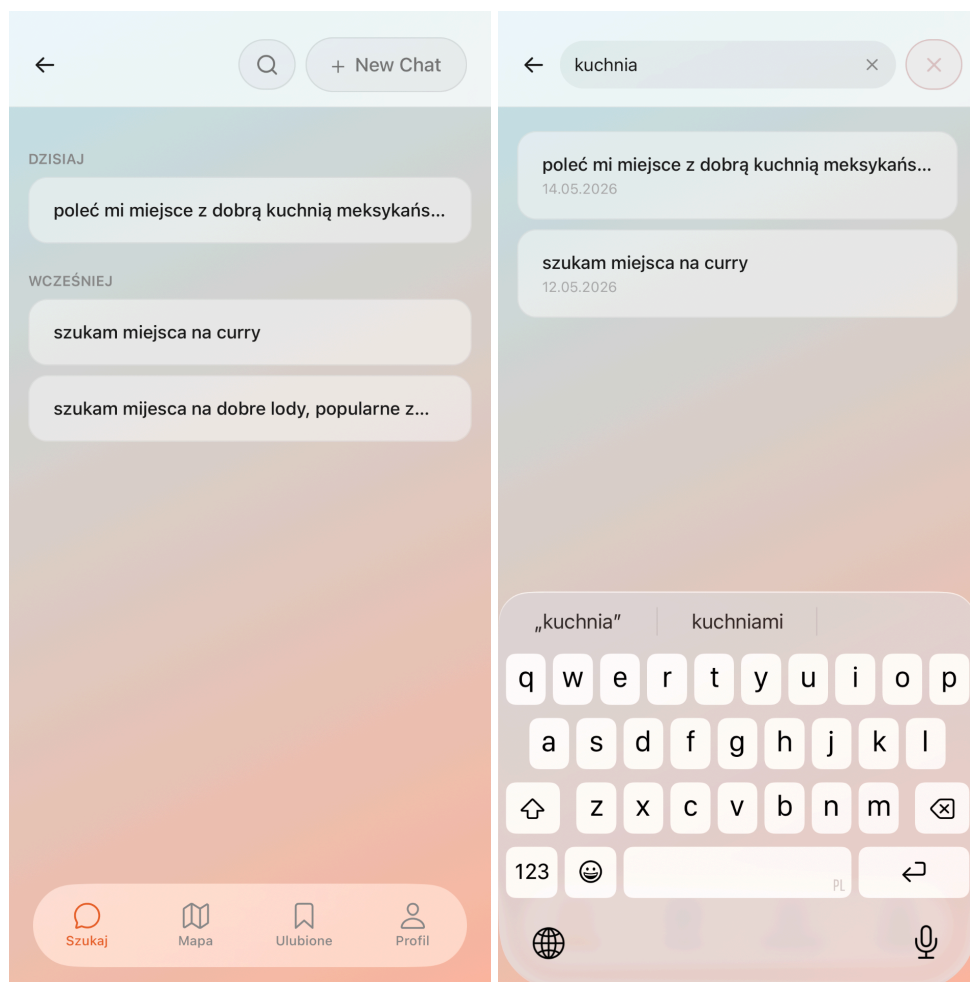
Screenshoty widoków



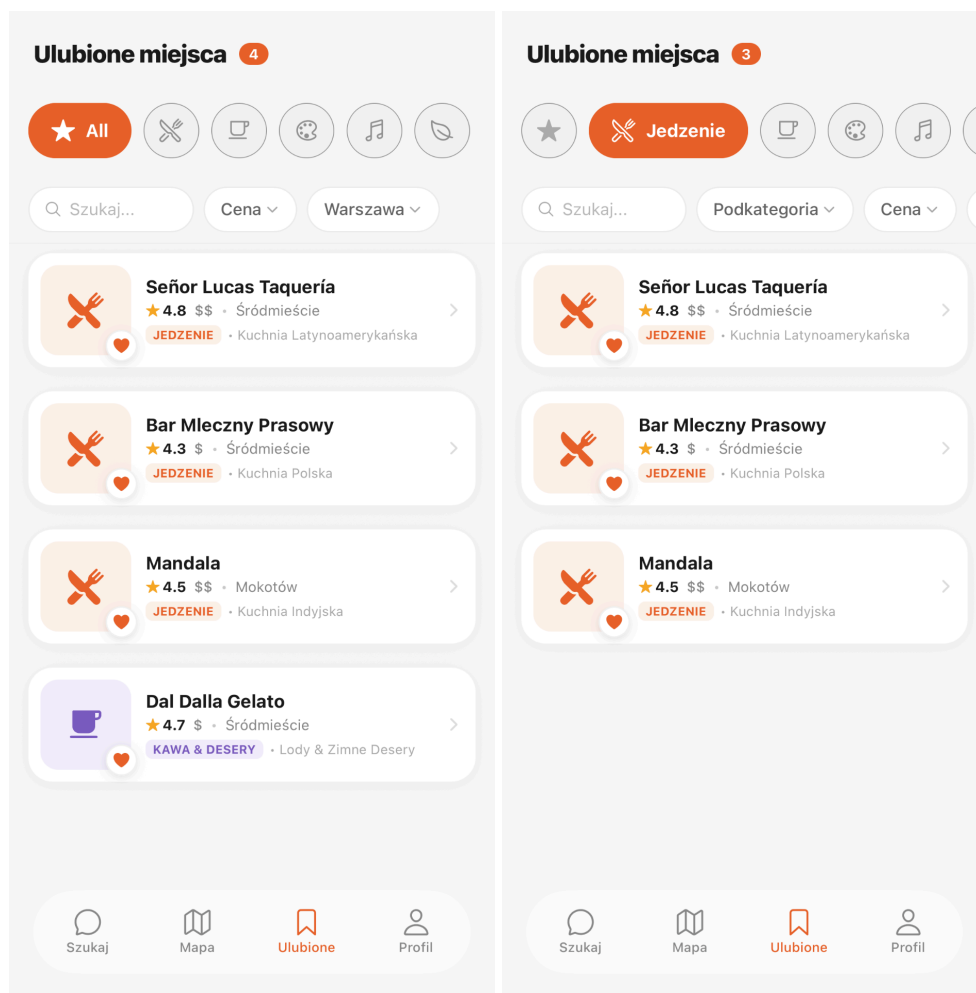
1. Ekran profilu



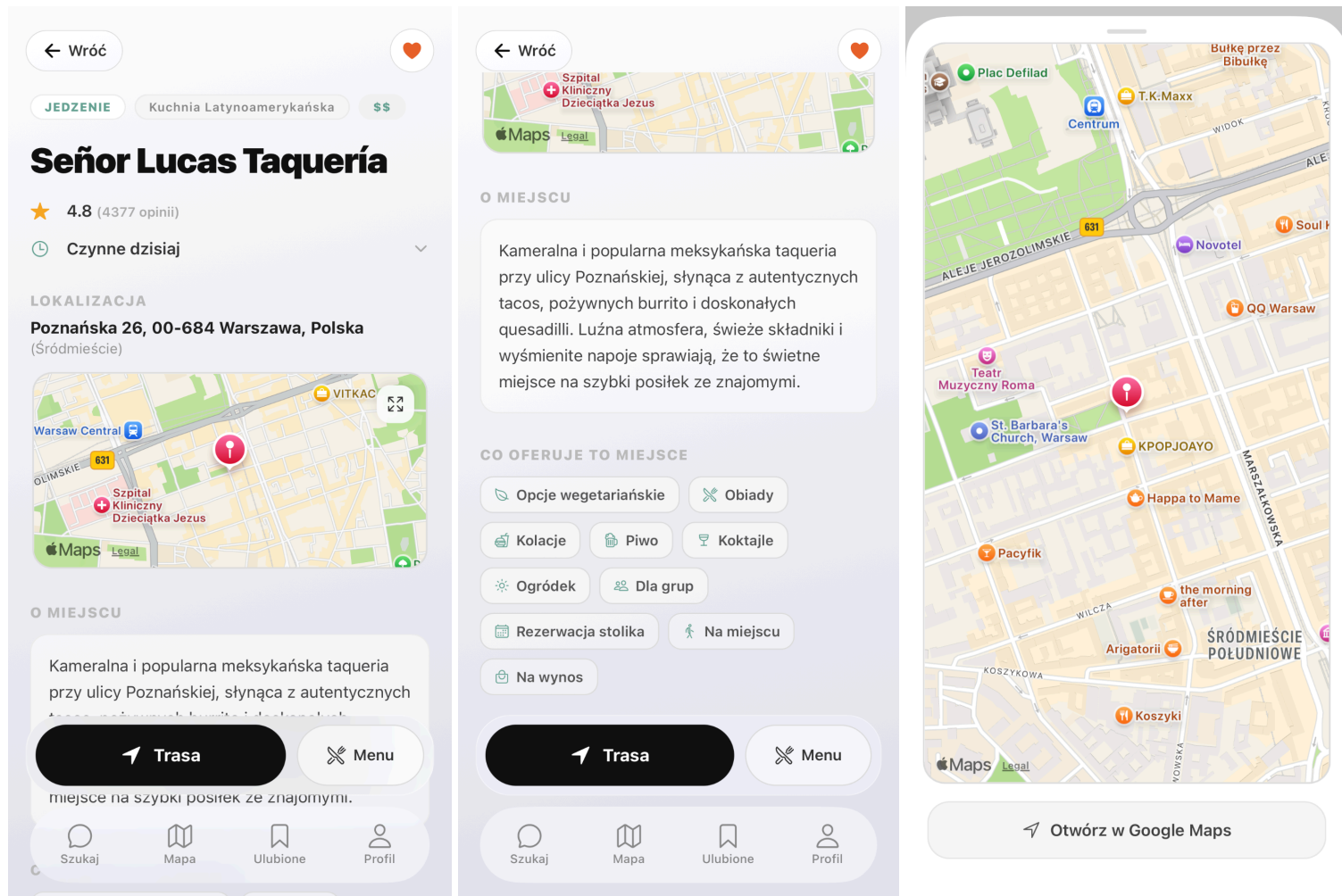
2. Ekran czatu — odpowiedź RAG ze sliderem poleconych miejsc



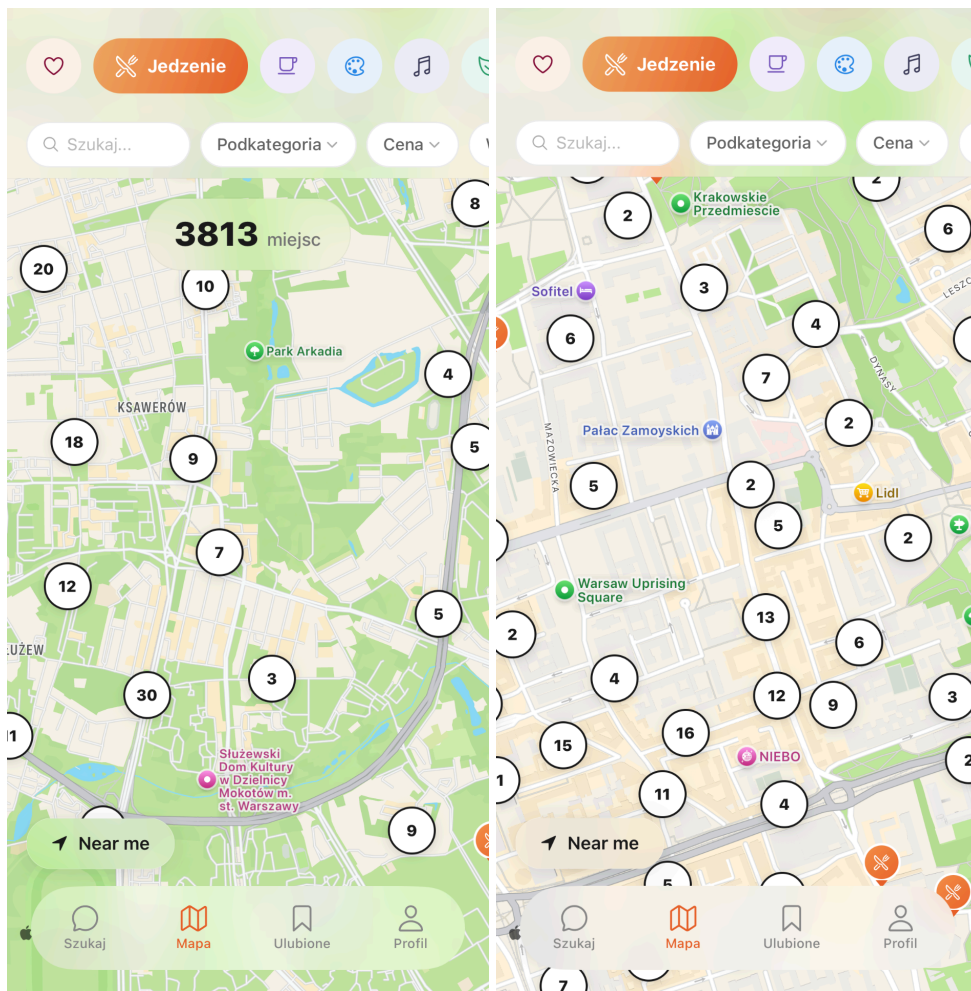
3. Historia sesji czatu



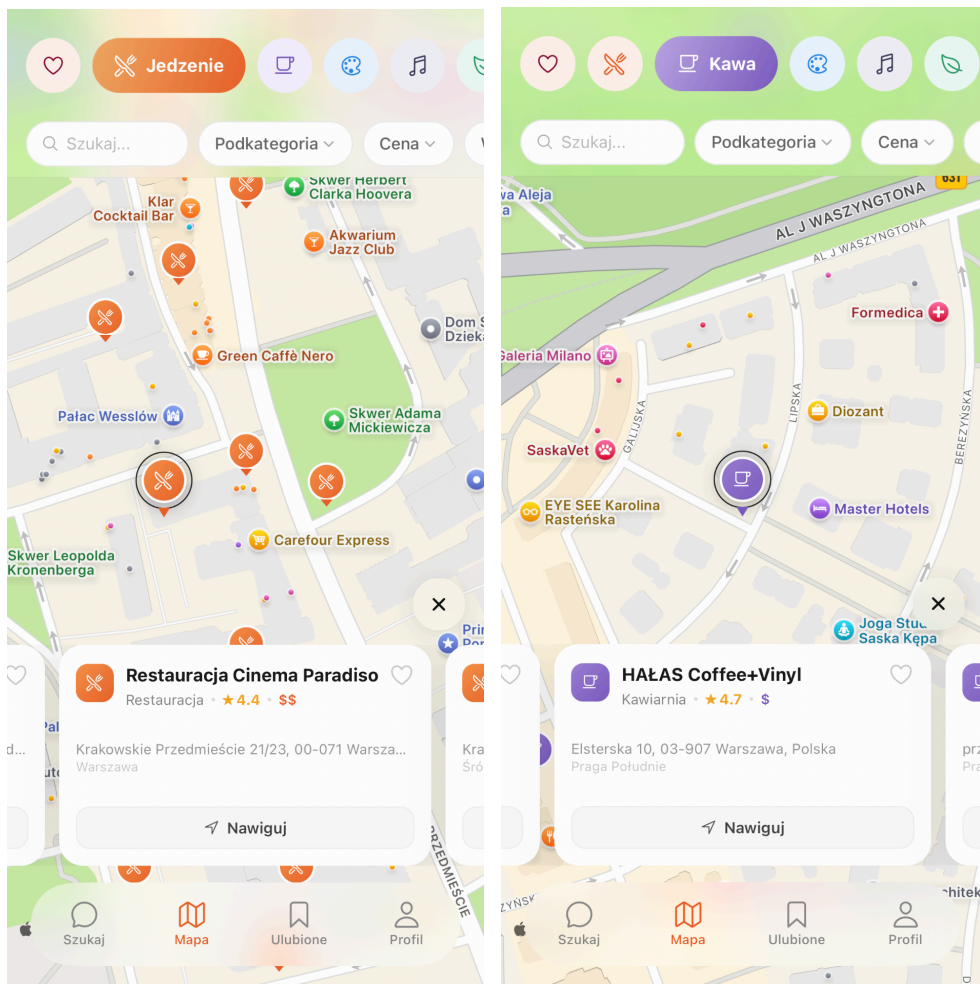
4. Ekran ulubionych miejsc



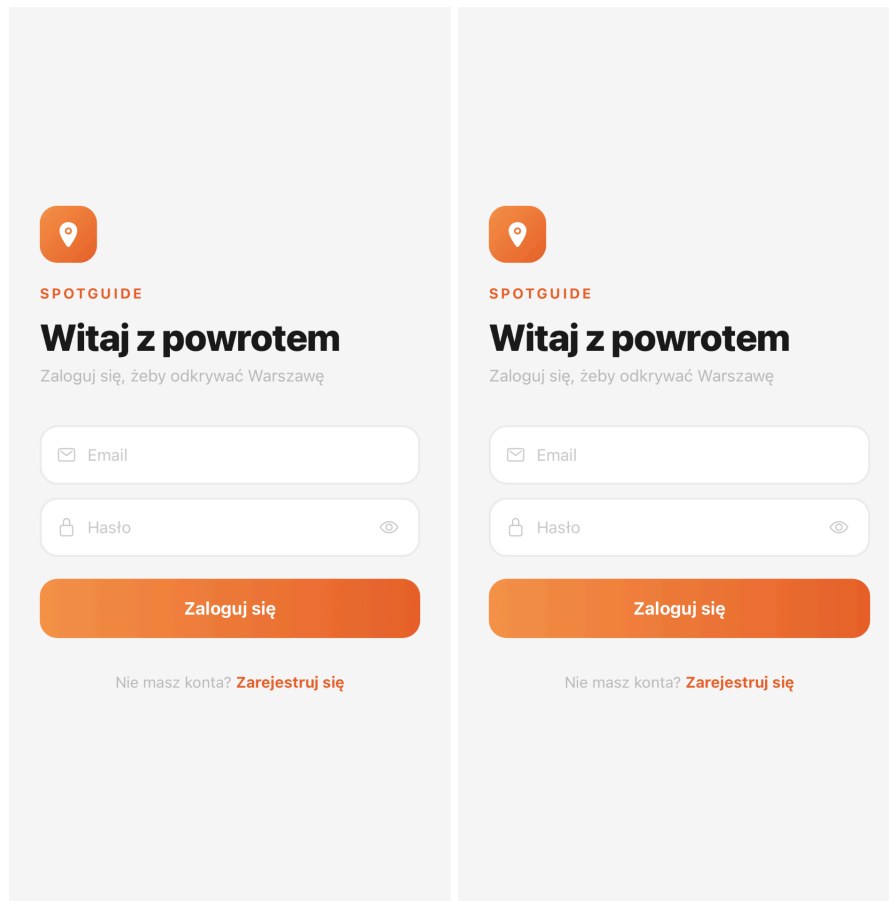
5. Ekran szczegółów miejsca



6. Mapa Warszawy z markerami i grupowanie



7. Slider kart miejsc na mapie



8. Ekran logowania i rejestracji (AWS Cognito)

12. Architektura infrastruktury i strategia wdrożenia chmurowego

Infrastruktura aplikacji zarządzana jest przez Terraform zgodnie z podejściem Infrastructure as Code (IaC). Wszystkie zasoby AWS — sieć VPC, baza danych RDS, klaster ECS, repozytorium ECR, autoryzacja Cognito oraz Load Balancer — są zdefiniowane w plikach konfiguracyjnych i tworzone automatycznie za pomocą jednego polecenia. Stan infrastruktury przechowywany jest zdalnie w zasobniku S3.

12.1 Struktura katalogów Terraform

```
terraform/
├── main.tf                # Root — definicje modułów i zależności
├── variables.tf           # Zmienne wejściowe
├── outputs.tf             # Outputy (cluster, service, repo names)
├── terraform.tfvars       # Wartości zmiennych (nie trafia do git)
├── terraform.tfvars.example # Szablon z placeholderami
└── modules/
    ├── networking/        # VPC, subnety, IGW, route tables
    ├── dns/               # Route 53 Hosted Zone
    ├── ssl/               # ACM Certificate + walidacja DNS
    ├── alb/               # Application Load Balancer
    ├── dns_records/       # Rekordy A wskazujące na ALB
    ├── ecr/               # Elastic Container Registry
    ├── ecs/               # ECS Cluster + Fargate Service
    ├── rds/               # PostgreSQL na RDS
    ├── cognito/           # User Pool, App Client
    └── ga/                # OIDC Provider + rola IAM dla CI/CD
```

12.2 Konfiguracja konta AWS

```
# Zainstaluj AWS CLI (jesli nie masz)
# https://docs.aws.amazon.com/cli/latest/userguide/install-cliv2.html

# Skonfiguruj credentials
aws configure
# AWS Access Key ID      -> z AWS Console -> IAM -> Users -> Security credentials
# AWS Secret Access Key -> generowany jednorazowo przy tworzeniu klucza
# Default region        -> eu-north-1
# Default output format -> json

# Zweryfikuj konfiguracje
aws sts get-caller-identity
```

12.3 Inicjalizacja i uruchomienie Terraform

Po skonfigurowaniu AWS CLI i uzupełnieniu terraform.tfvars należy uruchomić infrastrukturę etapami. Kolejność jest istotna ze względu na ręczną delegację NS do rejestratora domeny.

Stwórz bucket S3 na state Terraform

Bucket S3 musi istnieć zanim Terraform się zainicjalizuje — nie może stworzyć go sam (terraform init próbuje połączyć się z backendem już na etapie inicjalizacji). Ten krok wykonujesz tylko raz.

```
aws s3api create-bucket \
  --bucket spotguide-terraform-state \
  --region eu-north-1 \
  --create-bucket-configuration LocationConstraint=eu-north-1

aws s3api put-bucket-versioning \
  --bucket spotguide-terraform-state \
  --versioning-configuration Status=Enabled

aws s3api put-public-access-block \
  --bucket spotguide-terraform-state \
  --public-access-block-configuration
  "BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true"
```

12.4 Sieć — VPC i subnety

Fundamentem infrastruktury jest Virtual Private Cloud z blokiem adresowym 10.0.0.0/16. Dwa publiczne subnety (10.0.1.0/24 i 10.0.2.0/24) obsługują ALB i kontenery ECS — posiadają route do Internet Gateway, co umożliwia ruch wychodzący do OpenAI API. Dwa prywatne subnety (10.0.3.0/24 i 10.0.4.0/24) obsługują RDS — baza nie ma publicznego adresu IP i nie jest osiągalna spoza VPC.

ECS pozostaje w publicznych subnetach świadomie: przeniesienie do prywatnych wymagałoby NAT Gateway (~30 USD/miesiąc per strefę). Bezpieczeństwo kontenerów zapewniają security groups — ruch przychodzący dozwolony wyłącznie z security group ALB.

12.5 Domena i DNS — Route 53

Moduł dns tworzy Hosted Zone dla domeny aplikacji w Route 53. Domenę można pozyskać na dwa sposoby:

Opcja A — domena przez AWS Route 53

Jeśli domena jest zarejestrowana bezpośrednio w AWS (Route 53 → Registered Domains), rekordy NS zostaną skonfigurowane automatycznie i Hosted Zone będzie od razu aktywna — żadna ręczna konfiguracja DNS nie jest potrzebna.

Opcja B — domena u zewnętrznego rejestratora

Jeśli domenę jest u innego rejestratora (np. Namecheap, GoDaddy, OVH), po utworzeniu Hosted Zone Terraform wypisze cztery adresy NS. Należy wpisać je w panelu rejestratora jako Custom DNS. Ta operacja deleguje kontrolę nad DNS do Route 53 — propagacja trwa 15–30 minut.

W dalszej części dokumentacji opisana jest Opcja B.

12.6 Certyfikat SSL — ACM

Certyfikat HTTPS wystawiany przez AWS Certificate Manager (ACM) bezpłatnie, odnawia się automatycznie. Obejmuje domenę główną i wildcard (*.domena.com). Walidacja przez DNS — ACM generuje rekord CNAME który Terraform automatycznie dodaje do Route 53.

12.7 Application Load Balancer

ALB jest jedynym zasobem z publicznym adresem dostępnym z internetu. Listener port 80 → redirect 301 do HTTPS. Listener port 443 → TLS termination z certyfikatem ACM → Target Group → ECS (port 8000). Target Group skonfigurowana z target_type=ip (wymagane dla Fargate) i health checkiem /health co 30 sekund.

12.8 ECS Fargate — środowisko uruchomieniowe

Aplikacja backend działa jako kontener Docker na ECS Fargate. Task Definition: obraz Docker z ECR, 256 CPU units, 512 MB RAM, port 8000, zmienne środowiskowe wstrzykiwane przez ECS. IAM Execution Role: uprawnienia do pobrania obrazu z ECR i zapisu logów do CloudWatch.

12.9 Moduł ga — OIDC i rola IAM dla GitHub Actions

Moduł ga tworzy infrastrukturę potrzebną do bezpiecznego uwierzytelnienia GitHub Actions do AWS bez przechowywania kluczy dostępu. GitHub i AWS wymieniają tymczasowe tokeny przy każdym uruchomieniu pipeline (OIDC — OpenID Connect).

Poniższa instrukcja przedstawia kolejność tworzenia zasobów AWS opisanych w tym rozdziale.

```
cd terraform
cp terraform.tfvars.example terraform.tfvars # uzupełnij swoimi wartosciami
terraform init

# Krok 1: Hosted Zone + delegacja NS do rejestratora domeny
terraform apply -target=module.dns
# -> skopiuj 4 adresy NS z outputu do panelu rejestratora (Custom DNS)
# -> odczekaj 15-30 minut na propagacje DNS

# Krok 2: Certyfikat SSL (wymaga aktywnego DNS)
terraform apply -target=module.ssl
# -> walidacja ACM trwa 2-5 minut

# Krok 3: Cala pozostala infrastruktura (ECS, RDS, ALB, GitHub OIDC)
terraform apply

# Po apply - skopiuj output do GitHub Secrets:
terraform output github_actions_role_arn # -> AWS_ROLE_ARN (sekret GitHub)
```


12.10 Dockerfile — konfiguracja kontenera

Biblioteka timescale-vector wymaga psycopg2 kompilowanego ze źródeł — potrzebuje systemowych bibliotek PostgreSQL (libpq-dev, gcc).

```
FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y \
    libpq-dev \
    gcc \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ .
RUN adduser --disabled-password --gecos "" appuser
USER appuser

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--no-server-header"]
```

12.11 Zmienne środowiskowe — strategia dla środowisk

Lokalnie: cała konfiguracja w plikach .env (wykluczone z repo przez .gitignore).

Na produkcji: żaden plik .env nie istnieje — zmienne backendu wstrzykiwane przez ECS Task Definition (z terraform.tfvars), zmienne frontendu wbudowywane w binarny plik aplikacji podczas buildu EAS (z EAS Secrets). Warto jednak rozważyć migrację wrażliwych danych do usługi **AWS Secrets Manager**, który zapewnia bezpieczne szyfrowanie kluczem KMS oraz automatyczną rotację haseł bez konieczności restartu usług. Takie rozwiązanie centralizuje zarządzanie kluczami API i connection stringami, umożliwiając precyzyjną kontrolę dostępu za pomocą ról IAM zamiast wstrzykiwania zmiennych na sztywno do definicji zadań ECS.

Kolejność uzyskiwania wartości: identyfikatory Cognito (user_pool_id, app_client_id) są generowane przez Terraform i dostępne dopiero po terraform apply → uzupełnij terraform.tfvars → terraform apply ponownie → zapisz jako EAS Secrets dla buildu mobilnego.

12.12 Migracja bazy danych z lokalnego TimescaleDB na AWS RDS

Różnice między środowiskiem lokalnym a RDS

Tabela embeddings była hypertable TimescaleDB z indeksem diskann (vectorscale). Na RDS staje się zwykłą tabelą PostgreSQL15 z indeksem HNSW (pgvector). Pozostałe tabele (chat_sessions, chat_messages, saved_places) tworzone są jako puste — dane użytkowników powstają od zera na nowej instancji. Operator wyszukiwania wektorowego <=> pozostaje identyczny — kod backendu nie wymaga modyfikacji.

Element	Lokalna baza (Docker)	AWS RDS PostgreSQL 15
Silnik bazy	TimescaleDB	PostgreSQL 15 (bez TimescaleDB)
Tabela embeddings	Hypertable TimescaleDB	Zwykła tabela PostgreSQL
Indeks wektorowy	DiskANN (vectorscale)	HNSW (pgvector)
Operator <=>	Identyczny	Identyczny — backend bez zmian
Dane użytkowników	chat_sessions, chat_messages, saved_places	Tworzone puste — dane od zera na RDS

Plik schema_rds.sql

Schemat na RDS nie może być generowany przez pg_dump z lokalnej bazy — eksport zawiera odwołania do rozszerzeń TimescaleDB niekompatybilnych z RDS (m.in. create_hypertable, vectorscale). Plik schema_rds.sql został napisany ręcznie i zawiera czysty schemat gotowy do użycia na RDS:

Zawartość pliku	Opis
CREATE EXTENSION vector	Rozszerzenie pgvector — wyszukiwanie wektorowe
CREATE EXTENSION pg_trgm	Rozszerzenie do wyszukiwania pełnotekstowego (historia czatu)
CREATE TABLE places	Pełna struktura tabeli z 37 kolumnami — Google Place ID jako PK
CREATE TABLE embeddings	id UUID, metadata JSONB, contents TEXT, embedding VECTOR(1536)
CREATE TABLE chat_sessions	Pusta — dane użytkowników powstają od zera
CREATE TABLE chat_messages	Pusta — z FK do chat_sessions i indeksem GIN pg_trgm na kolumnie content
CREATE TABLE saved_places	Pusta — z FK do places i UNIQUE (user_id, place_id)
Indeks HNSW (zakomentowany)	Tworzony osobno po imporcie danych — patrz Krok 4

Plik schema_rds.sql znajduje się w katalogu database/ w repozytorium. Nie należy go generować przez pg_dump — używać wyłącznie ręcznie przygotowanej wersji.

Przegląd procedury

Migracja składa się z 5 kroków wykonywanych jednorazowo przy pierwszym wdrożeniu na RDS. Kroki 2–5 wykonywane są przez ECS Exec — bez otwierania portu RDS na zewnątrz. Kontener ECS działa wewnątrz VPC i ma bezpośredni dostęp do RDS przez sieć wewnętrzną (10.0.0.0/16).

Krok	Operacja	Gdzie wykonywana
1	Import schema_rds.sql — rozszerzenia i tabele	ECS Exec (wewnątrz VPC)
2	Eksport places z lokalnego Docker do CSV	Lokalnie (docker exec)
3	Eksport embeddings z lokalnego Docker do CSV	Lokalnie (docker exec)
4	Upload CSV na S3, import places i embeddings przez ECS	ECS Exec (wewnątrz VPC)
5	Utworzenie indeksu HNSW po imporcie danych	ECS Exec (wewnątrz VPC)

Krok 1 — Import schematu na RDS przez ECS Exec

Wejdź do kontenera ECS i uruchom rozszerzenia oraz schemat:

```
# Pobierz TASK_ID:
aws ecs list-tasks --cluster spotguide-cluster --region eu-north-1
```

```
# Wejdź do kontenera:
aws ecs execute-command \
  --cluster spotguide-cluster \
  --task <TASK_ID> \
  --container spotguide-api \
  --interactive \
  --command "/bin/sh"
```

```
# W kontenerze — włącz rozszerzenia i zaimportuj schemat:
python3 -c "
import psycopg2, os
conn = psycopg2.connect(os.environ['DATABASE_URL'])
conn.autocommit = True
cur = conn.cursor()
cur.execute('CREATE EXTENSION IF NOT EXISTS vector')
cur.execute('CREATE EXTENSION IF NOT EXISTS pg_trgm')
with open('/tmp/schema_rds.sql', 'r') as f:
    cur.execute(f.read())
print('Schemat OK')
"
```

Plik schema_rds.sql musi być dostępny w kontenerze. Skopiuj go przez S3 analogicznie jak CSV w Kroku 4, lub wgraj jako część obrazu Docker przed deploymentem.

Krok 2 — Eksport tabeli places z lokalnego Docker

Eksportujemy przez jawny SELECT — pomija child tables stworzone przez TimescaleDB:

```
docker exec timescaledb psql -U postgres -d warsaw_places -c \
  "\COPY (SELECT * FROM places)
  TO '/tmp/places.csv' WITH (FORMAT csv, HEADER true)"
```

```
docker cp timescaledb:/tmp/places.csv .
```

Krok 3 — Eksport tabeli embeddings z lokalnego Docker

Używamy jawnego SELECT zamiast COPY tabeli bezpośrednio — omija child table stworzoną przez TimescaleDB (_hyper_1_35_chunk):

```
docker exec timescaledb psql -U postgres -d warsaw_places -c \
```

```
"\COPY (SELECT id, metadata, contents, embedding FROM embeddings)
TO '/tmp/embeddings.csv' WITH (FORMAT csv, HEADER true)"
```

```
docker cp timescaledb:/tmp/embeddings.csv .
```

Krok 4 — Upload CSV na S3 i import przez ECS Exec

Pliki CSV kopiujemy do kontenera ECS przez S3 — bez otwierania portu RDS na zewnątrz:

```
# Upload na S3 (jednorazowo):
aws s3 cp places.csv s3://<TWOJ_BUCKET>/migration/places.csv
aws s3 cp embeddings.csv s3://<TWOJ_BUCKET>/migration/embeddings.csv
```

W kontenerze ECS:

```
# Pobierz pliki z S3:
aws s3 cp s3://<TWOJ_BUCKET>/migration/places.csv /tmp/places.csv
aws s3 cp s3://<TWOJ_BUCKET>/migration/embeddings.csv /tmp/embeddings.csv
```

```
# Import places:
python3 -c "
import psycopg2, os
conn = psycopg2.connect(os.environ['DATABASE_URL'])
cur = conn.cursor()
with open('/tmp/places.csv', 'r') as f:
    next(f)
    cur.copy_expert(
        'COPY places FROM STDIN WITH (FORMAT csv)', f)
conn.commit()
print('places OK')
"
```

```
# Import embeddings:
python3 -c "
import psycopg2, os
conn = psycopg2.connect(os.environ['DATABASE_URL'])
cur = conn.cursor()
with open('/tmp/embeddings.csv', 'r') as f:
    next(f)
    cur.copy_expert(
        'COPY embeddings (id, metadata, contents, embedding) FROM STDIN WITH
(FORMAT csv)', f)
conn.commit()
print('embeddings OK')
"
```

places importowane przed embeddings — tabela saved_places ma FK do places. Kolejność jest istotna.

Krok 5 — Indeks HNSW po imporcie danych

Indeks HNSW tworzony po załadowaniu danych — budowanie na gotowych rekordach jest szybsze niż podczas INSERTów. Parametry `m=16`, `ef_construction=64` optymalne dla ~6 000 rekordów:

```
python3 -c "
import psycopg2, os
conn = psycopg2.connect(os.environ['DATABASE_URL'])
conn.autocommit = True
```

```

cur = conn.cursor()
cur.execute('''
    CREATE INDEX idx_embeddings_hnsw ON public.embeddings
    USING hnsw (embedding public.vector_cosine_ops)
    WITH (m = 16, ef_construction = 64)
''')
print('Indeks HNSW OK')
"

```

Weryfikacja po migracji

Sprawdź liczbę zaimportowanych rekordów:

```

python3 -c "
import psycopg2, os
conn = psycopg2.connect(os.environ['DATABASE_URL'])
cur = conn.cursor()
cur.execute('SELECT COUNT(*) FROM places')
print('places:', cur.fetchone()[0])
cur.execute('SELECT COUNT(*) FROM embeddings')
print('embeddings:', cur.fetchone()[0])
cur.execute("SELECT indexname FROM pg_indexes WHERE tablename = 'embeddings'")
print('indeksy:', cur.fetchall())
"

```

Oczekiwany wynik: places i embeddings mają identyczną liczbę rekordów jak lokalna baza. Indeks idx_embeddings_hnsw widoczny na liście.

Po migracji usuń pliki CSV z S3 lub ogranicz dostęp do bucketa — pliki zawierają pełne dane miejsc wraz z embeddingami.

12.13 Backup bazy danych

Automatyczne backupy RDS z retention period 7 dni i point-in-time recovery. Okno backupu: 03:00–04:00 UTC. Przy bazie 20 GB storage backupów mieści się w darmowym limicie AWS (pierwsze 20 GB bezpłatne).

12.14 Monitoring i alarmy CloudWatch

Alarm	Zasób	Próg	Opis
spotguide-ecs-cpu-high	ECS	CPU > 80% przez 10 min	Kontener przeciążony — rozważ skalowanie
spotguide-ecs-memory-high	ECS	RAM > 85% przez 10 min	Ryzyko OOM kill kontenera
spotguide-rds-cpu-high	RDS	CPU > 80% przez 10 min	Baza przeciążona zapytaniem
spotguide-rds-storage-low	RDS	Wolne miejsce < 2 GB	Ryzyko zapełnienia dysku bazy
spotguide-alb-5xx-high	ALB	> 10 błędów 5xx/min	Backend zwraca błędy serwera
spotguide-alb-unhealthy-hosts	ALB	UnHealthyHosts Count > 0	Kontener ECS nie przechodzi health check

12.15 IAM — zarządzanie uprawnieniami

Credentials AWS CLI mają przypisaną politykę AdministratorAccess — akceptowalne na etapie developmentu jednej osoby. Wyjątkiem jest rola IAM dla GitHub Actions — ma ograniczone uprawnienia: tylko ECR PowerUser i ECS FullAccess. Pipeline nie ma dostępu do RDS, Route53, ACM ani żadnych innych zasobów. Jest to zgodne z zasadą least privilege.

12.16 Szacunkowe koszty infrastruktury

Poniższe szacunki dotyczą aktualnej konfiguracji AWS w regionie eu-north-1 (Stockholm) przy niskim ruchu (faza MVP / demo).

Usługa	Plan / tier	Koszt / mies.	Uwagi
AWS RDS PostgreSQL 15	db.t3.micro, 20 GB gp2	~15–18 USD	Single-AZ, backup 7 dni w cenie
AWS ECS Fargate	0.25 vCPU, 0.5 GB RAM	~8–12 USD	Jeden kontener, pay-per-use
AWS ALB	Application Load Balancer	~16–18 USD	Największy stały koszt — niezależny od ruchu
AWS ECR	Docker image storage	< 1 USD	Pierwsze 500 MB/mies. bezpłatne
AWS Cognito	< 50 000 MAU	0 USD	Bezpłatny do 50 000 aktywnych użytkowników/mies.
AWS CloudWatch	Logi ECS + alarmy	~2–4 USD	5 GB logów/mies. bezpłatnie
AWS Route53	1 hosted zone	~0.50 USD	Domena zarządzana zewnątrz
AWS ACM	Certyfikat TLS	0 USD	Bezpłatny przy użyciu z ALB
OpenAI API	gpt-4o-mini + text-embedding-3-small	~5–20 USD	Jedyny koszt zmienny — zależy od liczby zapytań
Google Maps SDK (mobile)	< 28 000 wywołań/mies.	0 USD	Bezpłatny limit Maps SDK for Mobile
GitHub Actions	CI/CD pipeline	0 USD	2 000 minut/mies. bezpłatnie
EAS Build (Expo)	Produkcyjny build mobilny	0 USD	Darmowy plan: 30 buildów/mies.

Szacowany całkowity koszt miesięczny: **~47–75 USD** przy niskim ruchu (MVP/demo). Dominuje ALB (~16–18 USD stały) i RDS (~15–18 USD stały). OpenAI to jedyny koszt rosnący z ruchem — każda wiadomość użytkownika to ~0.001–0.005 USD.

13. Środowisko developerskie a produkcyjne

Projekt funkcjonuje w dwóch odrębnych trybach, które różnią się konfiguracją, metodą uruchamiania i sposobem zarządzania sekretami.

13.1 Środowisko developerskie (lokalne)

```
# Terminal 1: baza danych
docker compose up -d

# Terminal 2: backend
cd backend/app && source .venv/bin/activate
uvicorn main:app --reload --port 8000 --host 0.0.0.0

# Terminal 3: frontend
cd frontend && npx expo start
```

13.2 Testowanie aplikacji mobilnej

Są trzy główne sposoby uruchamiania aplikacji podczas developmentu:

Expo Go — najszybszy start, bez kompilacji. Wystarczy zeskanować kod QR z terminala. Używaj do testowania UI i logiki aplikacji. Mapa (react-native-maps) crashuje w Expo Go nawet po zastosowaniu patcha — do testowania mapy wymagany jest development build.

Symulator / emulator — iOS Simulator dostępny przez Xcode (tylko Mac), Android Emulator przez Android Studio. Po uruchomieniu `npx expo start` naciśnij `i` dla iOS lub `a` dla Android. Nie wymaga fizycznego urządzenia ani konta Apple Developer.

Fizyczne urządzenie przez Xcode (tylko iOS) — szybka instalacja bez EAS. Otwórz projekt przez `.xcworkspace` (nie `.xcodeproj`):

```
open frontend/ios/frontend.xcworkspace
```

Podłącz iPhone kablem, wybierz urządzenie w Xcode i kliknij Run (▶). Przy pierwszym uruchomieniu: Settings → General → VPN & Device Management → zaufaj certyfikatowi dewelopera.

Development build przez EAS — wymagany do pełnego testowania mapy i natywnych modułów. Build trwa 10–15 minut w chmurze:

```
npm install -g eas-cli
eas login
eas build --profile development --platform ios    # wymaga płatnego konta Apple Developer
eas build --profile development --platform android
```

Po zakończeniu EAS wyśle link do pobrania pliku. Na iPhone przeciągnij .ipa na urządzenie w Xcode → Window → Devices and Simulators.

Zasada praktyczna: zacznij od Expo Go do testowania UI i logiki aplikacji. Przejdź na development build gdy testujesz mapę — mapa w Expo Go crashuje nawet po zastosowaniu patcha.

13.3 Build produkcyjny

Przed buildem produkcyjnym zapisz sekrety w chmurze EAS — zostaną automatycznie wstrzyknięte do aplikacji podczas budowania, bez potrzeby trzymania wartości produkcyjnych w repozytorium:

```
eas secret:create --scope project --name API_URL --value "https://spotguide.me"
eas secret:create --scope project --name COGNITO_USER_POOL_ID --value "eu-north-1_xxx"
eas secret:create --scope project --name COGNITO_APP_CLIENT_ID --value "xxx"
eas secret:create --scope project --name GOOGLE_MAPS_API_KEY --value "xxx"

eas build --profile production --platform all
```

13.4 Deploy backendu na AWS

Aktualizacje kodu backendu są wdrażane automatycznie przez GitHub Actions. Każdy push na gałąź main uruchamia pipeline który buduje nowy obraz Docker, wypycha go do ECR i wymusza restart serwisu ECS.

Konfiguracja pipeline opisana jest w rozdziale 13 (CI/CD). Pipeline można też uruchomić ręcznie bez zmiany kodu przez workflow_dispatch trigger: GitHub → Actions → Deploy Backend to ECS → Run workflow.

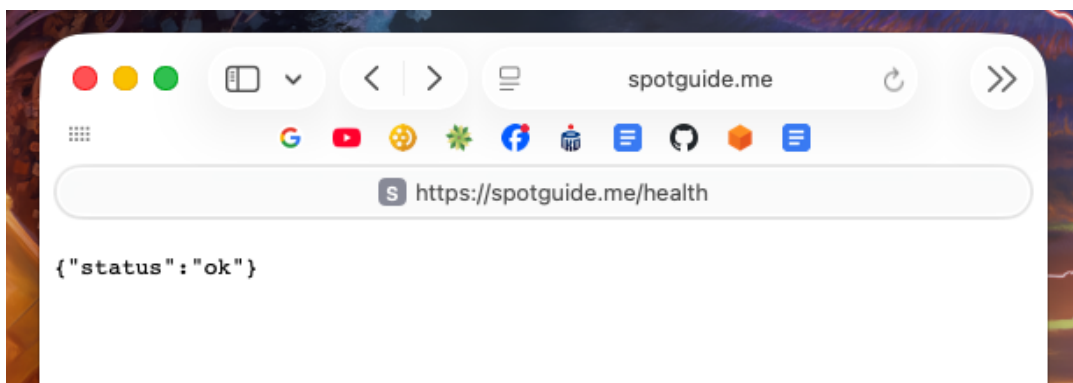
ECS automatycznie pobiera nowy obraz z ECR, uruchamia nowy kontener i po pozytywnym health checku (/health zwraca 200) kieruje ruch na nową wersję. Stary kontener jest wyłączany dopiero po potwierdzeniu zdrowia nowego — zero downtime deployment.

13.5 Weryfikacja wdrożenia

```
curl https://spotguide.me/health
# Oczekiwana odpowiedź: {"status": "ok"}

aws logs tail /ecs/spotguide-api --follow --region eu-north-1

aws ecs describe-services \
  --cluster spotguide-cluster \
  --services spotguide-api \
  --region eu-north-1
```




```
.2.84:2566 - "GET /health HTTP/1.1" 200 OK
2026-05-08T10:02:40.940000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.1.157:24658 - "GET /health HTTP/1.1" 200 OK
2026-05-08T10:03:10.110000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.2.84:14820 - "GET /health HTTP/1.1" 200 OK
2026-05-08T10:03:10.970000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.1.157:19714 - "GET /health HTTP/1.1" 200 OK
2026-05-08T10:03:34.895000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 2026-05-08 10:
03:34,895 - INFO - HTTP Request: GET https://cognito-idp.eu-north-1.amazonaws.com/eu-north-1_JfVnB
r3cb/.well-known/jwks.json "HTTP/1.1 200 OK"
2026-05-08T10:03:34.939000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.2.84:21236 - "GET /places/favourite-names HTTP/1.1" 200 OK
2026-05-08T10:03:36.988000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.2.84:21236 - "GET /places/favourites HTTP/1.1" 200 OK
2026-05-08T10:03:40.129000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.2.84:21240 - "GET /health HTTP/1.1" 200 OK
2026-05-08T10:03:41.001000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.1.157:3828 - "GET /health HTTP/1.1" 200 OK
2026-05-08T10:03:50.190000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.2.84:4330 - "GET /places/favourites HTTP/1.1" 200 OK
2026-05-08T10:03:51.271000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.2.84:32574 - "GET /places/all HTTP/1.1" 200 OK
2026-05-08T10:04:10.136000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.2.84:12438 - "GET /health HTTP/1.1" 200 OK
2026-05-08T10:04:10.714000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.2.84:12442 - "GET /sessions/ HTTP/1.1" 200 OK
2026-05-08T10:04:11.031000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.1.157:29282 - "GET /health HTTP/1.1" 200 OK
2026-05-08T10:04:12.020000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.2.84:12442 - "GET /sessions/06d896d7-510d-49fb-88bc-fe6d6c7a864f/messages HTTP/1.1" 200 OK
2026-05-08T10:04:32.930000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.2.84:18414 - "GET /sessions/ HTTP/1.1" 200 OK
2026-05-08T10:04:40.150000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.2.84:18420 - "GET /health HTTP/1.1" 200 OK
2026-05-08T10:04:41.045000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.1.157:6632 - "GET /health HTTP/1.1" 200 OK
2026-05-08T10:05:10.151000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.2.84:13228 - "GET /health HTTP/1.1" 200 OK
2026-05-08T10:05:11.070000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.1.157:18520 - "GET /health HTTP/1.1" 200 OK
2026-05-08T10:05:40.164000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.2.84:54504 - "GET /health HTTP/1.1" 200 OK
2026-05-08T10:05:41.097000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 INFO: 10.0
.1.157:31842 - "GET /health HTTP/1.1" 200 OK
2026-05-08T10:05:45.952000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 2026-05-08 10:
05:45,952 - INFO - HTTP Request: POST https://api.openai.com/v1/chat/completions "HTTP/1.1 200 OK"
2026-05-08T10:05:46.517000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 2026-05-08 10:
05:46,517 - INFO - HTTP Request: POST https://api.openai.com/v1/chat/completions "HTTP/1.1 503 Ser
vice Unavailable"
2026-05-08T10:05:46.518000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 2026-05-08 10:
05:46,518 - INFO - Retrying request to /chat/completions in 0.452867 seconds
2026-05-08T10:05:48.372000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 2026-05-08 10:
05:48,372 - INFO - HTTP Request: POST https://api.openai.com/v1/chat/completions "HTTP/1.1 200 OK"
2026-05-08T10:05:51.200000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 2026-05-08 10:
05:51,199 - INFO - HTTP Request: POST https://api.openai.com/v1/chat/completions "HTTP/1.1 200 OK"
2026-05-08T10:05:52.946000+00:00 ecs/spotguide-api/b3ec00eb3b4049bba7e154f9d3ab3536 2026-05-08 10:
05:52,946 - INFO - HTTP Request: POST https://api.openai.com/v1/embeddings "HTTP/1.1 200 OK"
```

14. CI/CD — automatyczne wdrożenia przez GitHub Actions

Każda zmiana kodu wypchnięta na gałąź main (w katalogu backend/) uruchamia pipeline który buduje obraz Docker, wypycha go do ECR i aktualizuje serwis ECS.

14.1 Jak działa pipeline

Krok	Nazwa	Co się dzieje
1	Checkout	GitHub pobiera kod z gałęzi main
2	Configure AWS	Uwierzytelnienie przez OIDC — bez przechowywania kluczy
3	Login to ECR	Docker loguje się do rejestru AWS
4	Set up Buildx	Docker Buildx ze sterownikiem docker-container (obsługa cache)
5	Build & Push	Buduje obraz z cache registry, taguje :latest i :SHA, wypycha do ECR
6	Deploy ECS	ECS pobiera nowy obraz, wykonuje rolling update
7	Wait stable	Pipeline czeka aż ECS potwierdzi zdrowie nowego kontenera

Cały proces: ~3–5 minut (pierwszy build bez cache: ~5–6 min). Błąd na dowolnym kroku zatrzymuje pipeline — poprzednia wersja działa bez przerwy.

14.2 OIDC — uwierzytelnienie do AWS

Pipeline używa OIDC zamiast kluczy AWS. Infrastruktura OIDC (provider i rola IAM) tworzona przez moduł ga w Terraform (patrz sekcja 11.10). Po terraform apply jedynym krokiem ręcznym jest:

1. GitHub → repo → Settings → Secrets and variables → Actions → New repository secret
2. Name: AWS_ROLE_ARN (wielkie litery — konwencja dla sekretów GitHub)
3. Value: skopiuj z: terraform output github_actions_role_arn

14.3 Cache Docker

Przy pierwszym buildzie lub po zmianie requirements.txt cache jest pusty — build trwa ~5–6 min. Przy kolejnych buildach bez zmiany requirements.txt warstwy pobierane z cache — build trwa ~1–2 min.

Sterownik docker-container w setup-buildx-action jest wymagany — domyślny sterownik docker nie obsługuje eksportu cache do registry.

14.4 Tagowanie i rollback

Każdy obraz otrzymuje dwa tagi: :latest (używany przez ECS) i :SHA_COMMITA (umożliwia rollback do konkretnej wersji). Rollback bez ponownego budowania:

```
# Wylistuj obrazy w ECR
aws ecr list-images --repository-name spotguide --region eu-north-1

# Przetaw tag :latest na wybrany SHA
MANIFEST=$(aws ecr batch-get-image \
  --repository-name spotguide \
  --image-ids imageTag=<SHA> \
  --query 'images[].imageManifest' --output text)

aws ecr put-image --repository-name spotguide --image-tag latest --image-manifest
"$MANIFEST"

aws ecs update-service --cluster spotguide-cluster --service spotguide-api \
  --force-new-deployment --region eu-north-1
```

15. Troubleshooting

Poniżej zestawienie problemów napotkanych podczas developmentu wraz z krokami naprawczymi. Każdy problem potwierdzony na fizycznym urządzeniu lub w środowisku lokalnym.

15.1 Problemy z buildem iOS (Xcode)

Crash przy uruchomieniu — User Script Sandboxing

Objaw: build kończy się błędem 'Sandbox: rsync.samba denied' lub skrypty postinstall nie wykonują się podczas kompilacji.

Przyczyna: Xcode od wersji 14 domyślnie włącza User Script Sandboxing, który blokuje wykonanie zewnętrznych skryptów (w tym patch-package) podczas fazy build.

Rozwiązanie: Xcode → projekt frontend → Build Settings → wyszukaj 'User Script Sandboxing' → ustaw na No.

Aplikacja nie uruchamia się na fizycznym urządzeniu — niezaufany certyfikat

Objaw: po instalacji na iPhone pojawia się komunikat 'Untrusted Developer' i aplikacja nie otwiera się.

Rozwiązanie: iPhone → Ustawienia → Ogólne → Zarządzanie VPN i urządzeniami → wybierz certyfikat dewelopera → Zaufaj.

Certyfikat wystawiony przez darmowe Apple Developer ID wygasa po 7 dniach — wymagana ponowna instalacja przez Xcode.

Błąd kompilacji — duplicate symbols (amazon-cognito-identity-js)

Objaw: build kończy się błędem 'duplicate symbol _AWSCognitoIdentity...' lub podobnym konfliktem symboli.

Przyczyna: jednoczesna obecność amazon-cognito-identity-js i aws-amplify w package.json — obie biblioteki zawierają te same natywne symbole.

Rozwiązanie: usuń amazon-cognito-identity-js z package.json — aws-amplify zawiera już tę funkcjonalność wewnętrznie.

```
npm uninstall amazon-cognito-identity-js
```

Projekt otwarty przez .xcodeproj zamiast .xcworkspace

Objaw: Xcode nie widzi zależności CocoaPods, build kończy się błędami 'module not found'.

Rozwiązanie: zawsze otwieraj projekt przez plik .xcworkspace, nie .xcodeproj.

```
open frontend/ios/frontend.xcworkspace
```

15.2 Problemy z mapą (react-native-maps)

Crash mapy przy zoom in/out — NSInvalidArgumentException

*Objaw: aplikacja crashuje przy wykonaniu zoom na mapie. W Xcode Console widoczny błąd: NSInvalidArgumentException *** -[__NSArrayM insertObject:atIndex:]: object cannot be nil*

Przyczyna: niekompatybilność react-native-maps 1.20.1 z Fabric renderer (Expo SDK 54 / React Native 0.81 z New Architecture).

Rozwiązanie: patch jest dołączony do projektu w patches/react-native-maps+1.20.1.patch i stosowany automatycznie po każdym npm install przez skrypt postinstall.

```
# Jeśli patch nie został zastosowany automatycznie:  
npm patch-package
```

Mapa działa poprawnie wyłącznie w development build lub production build — w Expo Go crash jest oczekiwany nawet po zastosowaniu patcha.

15.3 Problemy z backendem (FastAPI / Python)

Backend nie startuje — brakujące zmienne środowiskowe

Objaw: uvicorn uruchamia się ale natychmiast kończy z błędem ValidationError lub KeyError przy starcie.

Rozwiązanie: sprawdź czy plik backend/app/.env istnieje i zawiera wszystkie wymagane zmienne. Wzorzec w pliku .env.example w repozytorium.

```
OPENAI_API_KEY=sk-...  
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/warsaw_places  
COGNITO_USER_POOL_ID=eu-north-1_...  
COGNITO_APP_CLIENT_ID=...  
COGNITO_REGION=...  
ENVIRONMENT=dev
```

Błąd połączenia z bazą danych — psycopg2 connection refused

Objaw: każdy request do backendu kończy się błędem 500 z logiem 'could not connect to server: Connection refused'.

Rozwiązanie: sprawdź czy kontener TimescaleDB jest uruchomiony.

```
docker compose ps          # sprawdź status kontenera  
docker compose up -d       # uruchom jeśli zatrzymany  
docker compose logs -f     # logi kontenera przy problemach ze startem
```

Endpoint /chat/message zwraca 503 — OpenAI unavailable

Objaw: zapytania do czatu zwracają {'detail': 'AI service unavailable'} lub timeout po 30 s.

Możliwe przyczyny: nieprawidłowy klucz API, przekroczony limit RPM/TPM, niedostępność usługi OpenAI.

Rozwiązanie: sprawdź status OpenAI na status.openai.com, zweryfikuj klucz API i limity na platform.openai.com/usage.

```
# Test klucza API:  
curl https://api.openai.com/v1/models -H 'Authorization: Bearer $OPENAI_API_KEY'
```

Swagger (/docs) dostępny na produkcji

Objaw: endpoint https://spotguide.me/docs zwraca stronę dokumentacji zamiast 404.

Rozwiązanie: sprawdź zmienną środowiskową ENVIRONMENT w ECS Task Definition — musi być ustawiona na 'prod'. FastAPI warunkowo wyłącza /docs gdy ENVIRONMENT=prod.

```
aws ecs describe-task-definition --task-definition spotguide-api | grep  
ENVIRONMENT
```

Backend na ECS nie przechodzi health check

Objaw: alarm spotguide-alb-unhealthy-hosts w CloudWatch, serwis ECS restartuje kontener w pętli.

Możliwe przyczyny: błąd w kodzie uniemożliwiający start uvicorn, brakująca zmienna środowiskowa w ECS Task Definition, błąd połączenia z RDS.

Rozwiązanie: sprawdź logi ECS — najczęstsze przyczyny widoczne są w pierwszych sekundach startu kontenera.

```
aws logs tail /ecs/spotguide-api --follow --region eu-north-1
```

15.4 Problemy z autentykacją (AWS Cognito / Amplify)

Rejestracja lub logowanie nie działa — błąd konfiguracji Amplify

Objaw: logowanie kończy się błędem 'UserPool not configured' lub 'Invalid client_id'.

Rozwiązanie: sprawdź plik frontend/amplify-config.ts — wartości userPoolId i clientId muszą odpowiadać zasobom Cognito stworzonym przez Terraform.

```
# Pobierz wartości z Terraform output:  
terraform output cognito_user_pool_id  
terraform output cognito_app_client_id
```

Backend zwraca 401 mimo poprawnego tokenu

Objaw: requesty z aplikacji zwracają 401 mimo że użytkownik jest zalogowany i token wygląda poprawnie.

Możliwe przyczyny: niezgodność COGNITO_USER_POOL_ID lub COGNITO_APP_CLIENT_ID w zmiennych środowiskowych backendu, wygasły cache kluczy JWKS.

Rozwiązanie: zweryfikuj zmienne środowiskowe backendu i zrestartuj serwer żeby wyczyścić cache @lru_cache dla kluczy JWKS.

```
# Lokalnie — restart uvicorn  
# Na ECS — force new deployment:  
aws ecs update-service --cluster spotguide-cluster --service spotguide-api  
--force-new-deployment --region eu-north-1
```

Użytkownik zarejestrowany ale nie może się zalogować

Objaw: logowanie kończy się błędem 'UserNotConfirmedException'.

Rozwiązanie: konto wymaga weryfikacji email. Sprawdź skrzynkę pocztową i wpisz kod jednorazowy na ekranie weryfikacji. Możliwe ponowne wysłanie kodu przez ekran rejestracji.

15.5 Problemy z CI/CD (GitHub Actions)

Pipeline kończy się błędem na kroku 'Configure AWS'

Objaw: krok 'Configure AWS credentials' kończy się błędem 'Could not assume role' lub 'Token expired'.

Rozwiązanie: sprawdź sekret AWS_ROLE_ARN w ustawieniach repozytorium GitHub (Settings → Secrets and variables → Actions). Wartość musi dokładnie odpowiadać ARN roli IAM z outputu Terraform.

```
terraform output github_actions_role_arn
```

ECS nie pobiera nowego obrazu po deploymencie

Objaw: pipeline zakończył się sukcesem ale aplikacja na produkcji nadal używa starego kodu.

Rozwiązanie: sprawdź czy tag :latest w ECR wskazuje na nowy obraz. ECS używa tagu :latest — jeśli nie został nadpisany, restart kontenera nie przyniesie zmiany kodu.

```
aws ecr list-images --repository-name spotguide --region eu-north-1
# Wymuś nowy deployment:
aws ecs update-service --cluster spotguide-cluster --service spotguide-api
--force-new-deployment --region eu-north-1
```

15.6 Szybka diagnostyka — lista kontrolna

Przy niezidentyfikowanym problemie sprawdź w kolejności:

Krok	Polecenie / akcja	Oczekiwany wynik
1	curl https://spotguide.me/health	{'status': 'ok'} — backend działa
2	aws logs tail /ecs/spotguide-api --follow --region eu-north-1	Brak błędów — ostatnie logi bez ERROR/CRITICAL
3	docker compose ps	Status: Up — kontener TimescaleDB uruchomiony (lokalnie)
4	aws ecs describe-services --cluster spotguide-cluster --services spotguide-api	runningCount: 1, pendingCount: 0
5	Alarmy CloudWatch w AWS Console	Brak alarmów w stanie ALARM
6	npm patch-package (po npm install)	Patch react-native-maps zastosowany — brak błędu

16. Bezpieczeństwo - OWASP Web, API i Mobile Top 10

Bezpieczeństwo aplikacji weryfikowane jest zgodnie z czterema standardami OWASP: Web Top 10:2025, API Top 10:2023, Mobile Top 10:2024 oraz MASVS (Mobile Application Security Verification Standard) na poziomie L1. Każda kategoria pokryta jest automatycznymi testami integracyjnymi uruchamianymi przeciwko działającemu backendowi z prawdziwymi tokenami Cognito.

16.1 Metodologia testowania

Backend (Python/FastAPI) - zestaw dziesięciu plików pytest w backend/tests/, każdy odpowiadający jednej lub kilku kategoriom OWASP. Testy wymagają działającego serwera (uvicorn) oraz dwóch kont testowych w Cognito. Konfiguracja środowiska testowego: backend/tests/.env.test.

Frontend (React Native/Expo) - audyt zależności przez npm audit. Aplikacja mobilna nie eksponuje własnego API - testy logiki bezpieczeństwa realizowane są po stronie backendu.

Testy produkcyjne - plik test_production_smoke.py uruchamiany ręcznie z BASE_URL wskazującym na domenę produkcyjną. Weryfikuje warstwy niedostępne lokalnie: redirect HTTP->HTTPS, certyfikat TLS, wyłączenie /docs na produkcji, brak wildcard w CORS, obecność nagłówka HSTS.

Raport HTML - generowany poleceniem:

```
pytest tests/ -v --html=report.html --self-contained-html
```

Wynik końcowy: **138 passed, 3 skipped** w 5 sekund.

Logowanie zdarzeń - backend zapisuje zdarzenia bezpieczeństwa do pliku backend/app/app.log. Logi obejmują błędy autoryzacji (AUTH_FAILURE), wyjątki serwera i zdarzenia wszystkich endpointów.

16.2 OWASP Web Top 10:2025 - kategorie i pokrycie

Poniżej krótki opis każdej z dziesięciu kategorii OWASP Web Top 10:2025 wraz ze statusem pokrycia w projekcie.

A01:2025 - Broken Access Control

Najczęstsza kategoria podatności - użytkownik uzyskuje dostęp do zasobów innych użytkowników lub wykonuje operacje, do których nie ma uprawnień. Obejmuje IDOR (Insecure Direct Object Reference), brak weryfikacji własności zasobu przed jego modyfikacją oraz dostęp do endpointów bez uwierzytelnienia.

W projekcie: weryfikuje izolację sesji i ulubionych między użytkownikami, zwracanie 401 przy braku tokenu i 403 przy próbie dostępu do cudzego zasobu, oraz prawo do usunięcia własnych danych (RODO Art. 17).

A02:2025 - Security Misconfiguration

Błędna konfiguracja serwera, frameworka lub infrastruktury - ujawnione nagłówki technologiczne, zbyt szeroka polityka CORS, dostępna dokumentacja API na produkcji, domyślne hasła lub klucze. Często wynika z pominięcia konfiguracji bezpieczeństwa przy wdrożeniu.

W projekcie: weryfikuje nagłówki HTTP (X-Content-Type-Options, X-Frame-Options, HSTS, Server bez wersji), brak Swagger i ReDoc na produkcji, poprawność polityki CORS.

A03:2025 - Software Supply Chain Failures

Podatności wprowadzone przez zewnętrzne zależności - biblioteki z nieaktualnymi wersjami, złośliwe pakiety w łańcuchu dostawców, brak audytu zależności. W poprzednich edycjach OWASP kategoryzowane jako 'Vulnerable and Outdated Components'.

W projekcie: audyt pip-audit (backend) i npm audit (frontend). Szczegóły w sekcji 15.5.

A04:2025 - Cryptographic Failures

Słaba lub nieistniejąca kryptografia - przesyłanie danych wrażliwych w plain text, użycie słabych algorytmów (MD5, SHA1, alg:none w JWT), brak szyfrowania transmisji. Obejmuje również nieumyślne ujawnienie danych wrażliwych w odpowiedziach API.

W projekcie: weryfikuje że API nie zwraca connection stringów ani kluczy API, tokeny JWT używają RS256 (asymetryczny), komunikacja z produkcją wymaga HTTPS z ważnym certyfikatem CA-signed.

A05:2025 - Injection

Wstrzyk nielicencjonowanego kodu przez dane dostarczone przez użytkownika - SQL injection, command injection, XSS. Możliwe gdy aplikacja nie waliduje danych wejściowych przed ich przetworzeniem przez interpreter.

W projekcie: weryfikuje odporność na SQL injection w parametrach zapytań i polach body. Wszystkie zapytania do bazy używają parametryzowanych zapytań przez psycopg2. Walidacja typów i długości realizowana przez Pydantic.

A06:2025 - Insecure Design

Podatności wynikające z błędów na poziomie architektury i projektowania, nieimplementacji - brak izolacji danych między użytkownikami w schemacie bazy, deterministyczność operacji biznesowych, możliwość enumeracji zasobów po przewidywalnych identyfikatorach.

W projekcie: weryfikuje izolację ulubionych i sesji per użytkownik, deterministyczność toggle (pierwsza dodaje, druga usuwa, trzecia ponownie dodaje), brak możliwości enumeracji cudzych zasobów przez losowe UUID.

A07:2025 - Authentication Failures

Błędy w mechanizmach uwierzytelniania - akceptowanie fałszywych tokenów, brak weryfikacji podpisu JWT, nieprawidłowa obsługa wygaśnięcia sesji, brak ochrony przed brute force. Skutkuje możliwością podszywania się pod innych użytkowników.

W projekcie: weryfikuje że fałszywe tokeny, zmodyfikowane tokeny i tokeny z nieprawidłowym algorytmem odrzucane są z 401. Weryfikacja RS256 realizowana przez porównanie z kluczami publicznymi z JWKS endpoint Cognito.

A08:2025 - Software and Data Integrity Failures

Naruszenie integralności danych i kodu - podatność alg:none w JWT (podmiana algorytmu na brak weryfikacji), podpisywanie danych słabym kluczem, możliwość nadpisania tożsamości użytkownika przez dane z body requestu zamiast z zweryfikowanego tokenu.

W projekcie: weryfikuje odporność na atak alg:none (warianty: none, None, NONE), próby nadpisania user_id przez body requestu, podmiany payloadu JWT przy zachowaniu oryginalnego podpisu.

A09:2025 - Security Logging and Alerting Failures

Brak lub niewystarczające logowanie zdarzeń bezpieczeństwa - nielogowane próby nieautoryzowanego dostępu, brak alertów przy anomaliach, tokeny lub hasła w logach jako wrażliwe dane. Utrudnia wykrycie ataku i analizę post-mortem.

W projekcie: weryfikuje że błędy autoryzacji logowane są z tagiem AUTH_FAILURE, pliki logów nie zawierają pełnych tokenów JWT ani connection stringów, zdarzenia bezpieczeństwa są trwale zapisywane.

A10:2025 - Mishandling of Exceptional Conditions

Nieobsłużone wyjątki i edge case'y - serwer zwraca stack trace z wewnętrznymi szczegółami, crash przy nieoczekiwanych wartościach wejściowych, brak obsługi współbieżnych zadań modyfikujących ten sam zasób. Może ujawniać informacje o architekturze systemu.

W projekcie: weryfikuje obsługę nieprawidłowego JSON, ekstremalnych wartości liczbowych, nieistniejących zasobów, współbieżnych zadań toggle. Serwer nigdy nie zwraca niezaplanowanego 500 z wewnętrznymi szczegółami.

16.3 Mapowanie plików testowych na kategorie OWASP

Poniżej mapowanie dziesięciu plików testowych na kategorie OWASP Web Top 10:2025, API Top 10:2023 i Mobile Top 10:2024.

Plik testowy	Pokrywane kategorie	Obszar
test_a01_access_control.py	A01, API1, API5, M3, M6	Kontrola dostępu, izolacja danych między użytkownikami, IDOR
test_a02_security_misconfiguration.py	A02, API8, M5, M8	CORS, nagłówki HTTP, ekspozycja dokumentacji API
test_a03_supply_chain.py	A03, API9, M2	Audyt zależności, ekspozycja wersji technologii w odpowiedziach
test_a04_cryptographic.py	A04, M1, M10	Walidacja tokenów JWT, szyfrowanie transmisji, brak plain text secrets
test_a05_injection.py	A05, API10, M4	SQL injection w parametrach i polach body, walidacja Pydantic
test_a06_insecure_design.py	A06, API1, API6, M3, M6	IDOR, logika biznesowa, deterministyczny toggle ulubionych
test_a07_authentication.py	A07, API2, M1, M3	Weryfikacja tokenów Cognito RS256, odrzucanie fałszywych tokenów
test_a08_data_integrity.py	A08, API2, API3, M1, M10	Integralność JWT, ataki alg:none, podmiana payloadu, user_id z tokenu
test_a09_logging_alerting.py	A09, M9	Logowanie AUTH_FAILURE, brak tokenów i sekretów w logach
test_a10_exceptional_conditions.py	A10, API4, M4, M9	Obsługa wyjątkowych warunków, wartości brzegowe, współbieżność

16.4 Status pokrycia - OWASP Web Top 10:2025

ID	Kategoria	Plik testowy	Status	Uwagi
A01:2025	Broken Access Control	test_a01_access_control.py	Passed	Izolacja sesji i ulubionych, 401/403 poprawne
A02:2025	Security Misconfiguration	test_a02_security_misc onfiguration.py	Passed	Nagłówki HTTP, CORS, brak Swagger na prod
A03:2025	Software Supply Chain Failures	pip-audit + npm audit	Partial	1 podatność bez fixa (niskie ryzyko), 21 moderate frontend
A04:2025	Cryptographic Failures	test_a04_cryptographic.py	Passed	JWT RS256, HTTPS, brak sekretów w plain text
A05:2025	Injection	test_a05_injection.py	Passed	Zapytania parametryzowane, walidacja Pydantic
A06:2025	Insecure Design	test_a06_insecure_desi gn.py	Passed	IDOR przez UUID, deterministyczny toggle
A07:2025	Authentication Failures	test_a07_authenticatio n.py	Passed	Weryfikacja RS256 przez JWKS Cognito
A08:2025	Software and Data Integrity Failures	test_a08_data_integrity .py	Passed	Odpornosc na alg:none, user_id z tokenu
A09:2025	Security Logging and Alerting Failures	test_a09_logging_alerti ng.py	Passed	Logowanie AUTH_FAILURE, brak sekretów w logach
A10:2025	Mishandling of Exceptional Conditions	test_a10_exceptional_c onditions.py	Passed	Obsługa edge case'ow, brak 500 z internal details

16.5 OWASP API Top 10:2023 - pokrycie

ID	Kategoria	Status	Pokrycie przez testy
API1:2023	Broken Object Level Authorization	Pokryte	A01, A06 - izolacja sesji i ulubionych, losowe UUID
API2:2023	Broken Authentication	Pokryte	A07, A08 - weryfikacja JWT RS256, odrzucanie alg:none
API3:2023	Broken Object Property Level Authorization	Pokryte	A01, A08 - user_id wyłącznie z tokenu, nie z body
API4:2023	Unrestricted Resource Consumption	Pokryte	A10 - ekstremalne wartości, współbieżne zadania
API5:2023	Broken Function Level Authorization	Pokryte	A01, A06 - użytkownik B nie może usunąć zasobów użytkownika A

API6:2023	Unrestricted Access to Sensitive Business Flows	Pokryte	A06 - deterministyczny toggle, izolacja logiki biznesowej
API7:2023	Server Side Request Forgery	N/A	Backend nie konstruuje HTTP requestów z inputu użytkownika
API8:2023	Security Misconfiguration	Pokryte	A02 - nagłówki, CORS, wyłączenie Swagger, HSTS
API9:2023	Improper Inventory Management	Pokryte	A03 - brak legacy endpoints, brak ekspozycji wersji zależności
API10:2023	Unsafe Consumption of APIs	Pokryte	A05 - walidacja i sanitizacja danych wejściowych

API7:2023 (Server Side Request Forgery) nie jest pokryte testami - backend nie konstruuje requestów HTTP na podstawie danych dostarczonych przez użytkownika, co eliminuje powierzchnię ataku SSRF w tej architekturze.

16.6 Audyt zależności (A03 - Supply Chain)

Backend - pip-audit

Audyt przeprowadzono poleceniem pip-audit. Wynik końcowy: 6 znanych podatności w 5 pakietach.

Pakiet	Wersja	CVE	Fix	Status
aiohttp	3.13.3	CVE-2026-34513 i 9 innych	3.13.4	Zaktualizowano
anthropic	0.86.0	CVE-2026-34450, CVE-2026-34452	0.87.0	Zaktualizowano
pygments	2.19.2	CVE-2026-4539	2.20.0	Zaktualizowano
urllib3	2.6.3	CVE-2026-44431	2.7.0	Zaktualizowano do 2.7.0
urllib3	2.6.3	CVE-2026-44432	2.7.0	Zaktualizowano do 2.7.0
diskcache	5.6.3	CVE-2025-69872	brak	Brak fixa - zależność pośrednia instructor

! CVE-2025-69872 w diskcache 5.6.3 - zależność pośrednia biblioteki instructor (klient OpenAI). W projekcie diskcache nie przetwarza danych zewnętrznych użytkowników - ryzyko niskie. Podatność zostanie rozwiązana gdy instructor zaktualizuje zależność.

Frontend - npm audit

Audyt przeprowadzono poleceniem npm audit. Wynik: 6 podatności o poziomie moderate, wszystkie w narzędziach build-time lub zależności pośrednich. Żadna nie wpływa na aplikację produkcyjną.

Pakiet	Poziom	Problem	Dlaczego nie naprawiono
markdown-it < 12.3.2	moderate	Uncontrolled Resource Consumption	Brak fixa bez zmiany react-native-markdown-display
postcss < 8.5.10	moderate	XSS via Unescaped </style>	Fix = downgrade Expo do v49 (breaking change)

[INFO] Wszystkie 6 podatności npm dotyczą narzędzi build-time i zależności pośrednich - nie wpływają na bezpieczeństwo aplikacji produkcyjnej uruchomionej na urządzeniu użytkownika.

[BŁĄD] NIE UŻYWAĆ: npm audit fix --force — komenda ta wymusza breaking changes bez potwierdzenia i może drastycznie pogorszyć stan bezpieczeństwa projektu.

16.7 OWASP Mobile Top 10:2024 - pokrycie

ID	Kategoria	Krytyczność	Status	Uwagi
M1	Improper Credential Usage	Wysoka	OK	Tokeny JWT przez Amplify, weryfikacja RS256
M2	Inadequate Supply Chain Security	Wysoka	Partial	1 podatność bez fixa, 6 moderate frontend
M3	Insecure Authentication/Authorization	Wysoka	OK	Izolacja sesji i ulubionych, tokeny Cognito RS256
M4	Insufficient Input/Output Validation	Wysoka	OK	Walidacja Pydantic, SQL injection, obsługa błędnych inputów
M5	Insecure Communication	Wysoka	OK	HTTP->HTTPS redirect, certyfikat ACM, nagłówki bezpieczeństwa
M6	Inadequate Privacy Controls	Wysoka	OK	Prawo do usunięcia, brak cross-user leakage, ulubione per user
M7	Insufficient Binary Protections	Niska	N/A	Wymaga narzędzi reverse engineering (Frida, objection)
M8	Security Misconfiguration	Wysoka	OK	Nagłówki HTTP, CORS, brak Swagger na produkcji
M9	Insecure Data Storage	Wysoka	OK	Brak credentials w odpowiedziach, brak stack trace w logach
M10	Insufficient Cryptography	Wysoka	OK	JWT RS256, HTTPS TLS 1.2+, brak wrażliwych danych w plain text

16.8 OWASP MASVS L1 - weryfikacja

MASVS L1 to baseline bezpieczeństwa mobilnego - wymagania które powinna spełniać każda aplikacja mobilna. Poziomy L2 i R (certificate pinning, obfuskacja kodu) stosowane są głównie w bankowości i aplikacjach rządowych.

Wymaganie MASVS	Opis	Status	Implementacja
MASVS-STORAGE-1	Wrażliwe dane nie są przechowywane lokalnie	OK	Tokeny JWT przechowywane przez Amplify w AsyncStorage
MASVS-STORAGE-2	Brak wrażliwych danych w logach aplikacji	OK	Brak print() z tokenami/kluczami w kodzie produkcyjnym
MASVS-CRYPTO-1	Silna kryptografia	OK	JWT RS256 (RSA-2048), TLS 1.2+ przez ALB z ACM
MASVS-AUTH-1	Autoryzacja po stronie serwera	OK	Każdy endpoint wymaga weryfikacji JWT przez Cognito JWKS
MASVS-AUTH-2	Sesja unieważniania przy wylogowaniu	OK	signOut() Amplify unieważnia tokeny i czyści AsyncStorage
MASVS-NETWORK-1	Szyfrowanie transmisji (TLS)	OK	HTTPS enforced przez ALB, HTTP redirect 301
MASVS-NETWORK-2	Weryfikacja certyfikatu TLS	OK	Certyfikat ACM, weryfikacja przez system operacyjny
MASVS-PLATFORM-1	Brak nadmiarowych uprawnień	OK	Lokalizacja: tylko foreground gdy mapa aktywna
MASVS-CODE-1	Brak wrażliwych danych w kodzie źródłowym	OK	Sekrety przez EAS Secrets / Terraform - nie w repo
MASVS-RESILIENCE-1	Ochrona przed inżynierią wsteczna	Partial	Brak certificate pinning i obfuskacji - akceptowalne dla L1

16.9 Testy produkcyjne i poprawki wdrożeniowe

Po wdrożeniu przeprowadzono testy smoke przeciwko środowisku produkcyjnym (backend/tests/test_production_smoke.py). Wynik: 7 z 7 testów przechodzi.

Weryfikowane obszary: redirect HTTP->HTTPS (ALB listener port 80 -> 301), ważny certyfikat TLS (CA-signed przez ACM), wyłączenie /docs i /redoc na produkcji, brak wildcard w nagłówku Access-Control-Allow-Origin, obecność nagłówka Strict-Transport-Security (HSTS).

W trakcie wdrożenia wykryto i naprawiono trzy problemy:

Ekspozycja Swagger - /docs było dostępne publicznie na produkcji. Naprawiono przez warunkowe wyłączenie w FastAPI() na podstawie zmiennej środowiskowej ENVIRONMENT=prod wstrzykiwanej przez ECS Task Definition.

Brak nagłówka HSTS - ALB nie dodaje nagłówka Strict-Transport-Security automatycznie. Naprawiono przez middleware w main.py (Strict-Transport-Security: max-age=31536000; includeSubDomains). Bez tego nagłówka możliwy był atak SSL stripping.

Ekspozycja wersji uunicorn - nagłówek Server: uunicorn ujawniał technologie serwera. Naprawiono przez flagę --no-server-header w CMD Dockerfile oraz nadpisanie nagłówka Server: spotguide w middleware.

16.10 Wnioski z testów bezpieczeństwa

Wynik końcowy 138 passed, 3 skipped w 5 sekund potwierdza że podstawowy poziom bezpieczeństwa jest osiągnięty. Poniżej kluczowe obserwacje z przebiegu testów.

Co działało poprawnie od początku

Autoryzacja i izolacja danych użytkowników (A01, A06) działały poprawnie od pierwszego uruchomienia testów - weryfikacja JWT przez Cognito JWKS, sprawdzanie własności sesji przed odczytem i izolacja ulubionych per user_id były zaimplementowane prawidłowo. Testy A07 i A08 (fałszywe tokeny, alg:none, podmiana payloadu) przeszły od razu, co potwierdza że biblioteka weryfikacji JWT w FastAPI jest skonfigurowana prawidłowo. Testy produkcyjne (test_production_smoke.py) były jedynym zestawem, który wymagał poprawek po stronie infrastruktury, nie kodu.

Problemy wykryte i naprawione w trakcie testowania

Główny pattern błędów: backend zwracał 500 zamiast właściwych kodów HTTP (403, 404, 422). Nie był to problem z logiką bezpieczeństwa, ale z propagacją wyjątków z warstwy repozytorium do routera. Konkretnie:

DELETE /sessions/{id} -> 500 zamiast 403 - metoda delete_session w ChatRepository rzucała PermissionError gdy sesja należała do innego użytkownika, ale router nie obsługiwał tego wyjątku i zwracał ogólny 500. Naprawiono przez dodanie except PermissionError w routerze.

POST /places/favourites/{id}/toggle -> 500 zamiast 422 - metoda toggle_favourite w PlacesRepository nie obsługiwała ForeignKeyViolation z PostgreSQL gdy place_id nie istniał w tabeli places (testy używały losowych UUID). Naprawiono przez obsługę pg_errors.ForeignKeyViolation i podniesienie ValueError, które router konwertuje na 422.

Współbieżny toggle -> 500 - race condition przy równoczesnych requestach na ten sam (user_id, place_id). Naprawiono przez obsługę DeadlockDetected i UniqueViolation w repozytorium.

Nierozwiązane podatności

CVE-2025-69872 w diskcache 5.6.3 (zależność pośrednia biblioteki instructor) pozostaje bez fixa - brak dostępnej wersji instructor z zaktualizowaną zależnością. Ryzyko ocenione jako niskie: diskcache w projekcie nie przetwarza danych zewnętrznych użytkowników. Podatność zostanie rozwiązana automatycznie przy kolejnej aktualizacji instructor.

6 podatności npm o poziomie moderate w zależnościach frontendu (markdown-it, postcss, uuid) nie może być naprawionych bez wprowadzenia breaking change w Expo SDK lub aws-amplify. Wszystkie dotyczą narzędzi build-time - nie wpływają na bezpieczeństwo aplikacji produkcyjnej.

Ogólna ocena

Architektura bezpieczeństwa projektu jest solidna: JWT RS256 przez Cognito, parametryzowane zapytania SQL przez psycopg2, Pydantic jako warstwa walidacji, izolacja danych przez user_id z tokenu. Problemy wykryte w testach były implementacyjne (brakujące except bloki), nie architektoniczne - co jest istotna różnica przy ocenie dojrzałości systemu bezpieczeństwa.

16.11 Podsumowanie

Standard	Zakres	Status	Uwagi
OWASP Web Top 10:2025	A01-A10 - bezpieczeństwo aplikacji webowych	9/10	A03 partial - 1 podatność bez fixa (niskie ryzyko)
OWASP API Top 10:2023	API1-API10 - bezpieczeństwo API	9/10	API7 N/A - SSRF nieistotny dla tej architektury
OWASP Mobile Top 10:2024	M1-M10 - bezpieczeństwo mobilne	9/10	M7 poza zakresem - wymaga narzędzi reverse engineering
OWASP MASVS L1	Baseline bezpieczeństwa mobilnego	Passed	Wszystkie wymagania L1 spełnione

Projekt pokrywa cztery standardy bezpieczeństwa: OWASP Web Top 10:2025, OWASP API Top 10:2023, OWASP Mobile Top 10:2024 i OWASP MASVS L1. Wynik końcowy testów automatycznych: 138 passed, 3 skipped w 5 sekund.